

# KAFRIADA CORE

System and solution architecture for the Jigawa State pilot — a free athlete identity register with a permanent KUID, a paid verification tier, and a closed-loop ledger that takes money in and never lets it out.

| PREPARED FOR                 | PILOT WINDOW           | SCOPE                  | TARGET SCALE              | STATUS              |
|------------------------------|------------------------|------------------------|---------------------------|---------------------|
| CTO, KowaGuru Technology Ltd | 8 weeks · 3 developers | Jigawa · 27 LGAs · 18+ | 5,000 athletes · 81 clubs | Draft v1 for review |

00

## The stack decision

You told me to ignore the pinned stack and choose. I chose the one you already have — here is why, because the reasoning matters more than the answer.

You asked whether FastAPI or Nuxt belong here. Short answer: **no, and I would not change what the Build Spec already committed to.**

The decisive factor is not language quality. FastAPI is excellent, and for a system that is mostly money movement, state machines and batch jobs, Python is a genuinely good fit. The decisive factor is **how many things three developers have to hold in their heads for eight weeks.** A FastAPI + Next.js split means two deploy units, two dependency trees, two CI pipelines, a CORS configuration, cookie-domain configuration for cross-origin sessions, request validation duplicated on both sides of the wire, and two places to look when something breaks at 11pm during a registration drive in Birnin Kudu. At 5,000 athletes that split buys nothing in performance, scale or team parallelism.

There is also a non-technical cost worth weighing: **the Next.js monolith is already in the Build Spec the CEO signed off.** Reversing an architecture decision that sits in a board-level document, for no measurable pilot benefit, spends credibility you will want later for decisions that actually matter.

Nuxt is a clearer no. There is no stated Vue investment anywhere in KAFRIADA, and adopting it would put three frontend frameworks inside one small organisation.

## What I am recommending

| LAYER           | CHOICE                                                                                                                                          | WHY THIS, AND NOT THE ALTERNATIVE                                                                                                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App             | Next.js 15 App Router, TypeScript. UI and API routes in one repo, one deploy.                                                                   | One language, one deploy, one auth surface. Server Components render the public profile as plain HTML — which matters more than it looks, see the Opera Mini problem in §1.8.                                                             |
| Worker          | A second always-on Node process. <i>Same repository, same code, different endpoint.</i>                                                         | My one addition to the Build Spec. Cron expiry, payment reconciliation, image derivatives and SMS retries must not run inside a web request. Two deploy units, one codebase; roughly \$7/month.                                           |
| Database        | PostgreSQL 16, managed, with point-in-time recovery. Prisma for ordinary access, <b>raw SQL for the counter, the ledger and reconciliation.</b> | Prisma cannot express <code>SELECT ... FOR UPDATE</code> or an atomic increment-and-return. Those three places are where correctness lives, so they are hand-written SQL reviewed by two people. Everything else gets the ergonomic path. |
| Region          | App and database both in <b>Frankfurt (eu-central)</b> , same availability zone.                                                                | Nigeria to Frankfurt is ~90–130ms; to US-East ~160–200ms. On a 500ms p95 budget, that gap is a quarter of the budget spent on geography. Never split app and DB across regions.                                                           |
| Edge            | Cloudflare in front of everything; public profile pages cached at the edge.                                                                     | Cloudflare has a Lagos point of presence. A match-day QR scan should be answered in Lagos and never reach your origin.                                                                                                                    |
| Everything else | Paystack, Termii, Cloudflare R2, Resend, Sentry.                                                                                                | No change. These are correct as specified.                                                                                                                                                                                                |

#### THE ONE CONDITION THAT FLIPS THIS

If your three developers are **Python-primary and weak in TypeScript**, invert it: FastAPI + SQLAlchemy + Alembic for the API, Next.js for the UI, both under one parent domain ( `kafriada.ng` and `api.kafriada.ng` ) so session cookies stay same-site. Everything else in this document — module boundaries, state machines, flows, security floor, rollout — is unchanged, because none of it depends on the language. The delta is about four days of setup and a permanently higher operational load. **Team fluency beats architectural elegance in an eight-week pilot.** Tell me which way the team leans and I will restate §3 and §4 in FastAPI terms.

## 01 Challenging the spec

Fifteen problems, ordered by what they will cost you. Read this before the design sections — several of them change the design.

### A. Internally inconsistent, or will bite

#### 1 — THE KUID MINTS UNDER A LOCK THAT YOUR PASSWORD HASH IS HOLDING

The spec mints the KUID with `SELECT ... FOR UPDATE` on a `kuid_counters` row keyed `(year, state_code)`, inside the Stage-1 registration transaction. In the pilot there is *one state*. So **every registration in Jigawa contends on a single database row**.

Survivable — but only if the lock is held for microseconds. If that same transaction also performs argon2id password hashing (100–300ms of CPU, by design), you are holding a global registration lock for a third of a second and your ceiling becomes three registrations per second. A hall with 200 people signing up would queue for over a minute and start timing out.

Two corrections, cheap now and painful later:

- **Hash the password before the transaction opens.** Never do CPU work inside a lock.
- **Replace the read-modify-write with one atomic statement.** `INSERT ... ON CONFLICT DO UPDATE SET next = counters.next + 1 RETURNING next` takes the row lock for the duration of a single statement rather than the duration of your transaction logic. Full proof in §4a.

#### 2 — NOTHING IN THE SPEC SAYS WHAT MAKES AN ATHLETE *UNIQUE*

This is the deepest unstated assumption in the document and the one I would fix first. A permanent identity register needs an anchor — the thing that says "this human already has a KUID, do not issue a second one." The spec never names it. You have three candidates:

- **Phone number.** Practical, verifiable by OTP, already in the design. But numbers change and get shared, and one person can hold four SIMs.
- **NIN.** Strong, but collecting and storing National Identification Numbers pulls NIMC regulation onto you, and verifying one properly means a paid lookup you have not budgeted. Storing NIN *without* verifying it is the worst of both worlds — the compliance duty with none of the benefit.
- **Human judgement at Stage-2 review.** Which only ever covers the 4% who pay.

**Recommendation for the pilot: the phone number is the anchor.** One verified phone, one KUID, enforced by a unique constraint in the database rather than by application logic. Changing an athlete's phone becomes an admin action with a reason and an audit row. Collect date of birth; do not collect NIN in the pilot; write "NIN linkage — Phase 2" into the roadmap so the CEO reads it as sequenced rather than missing.

#### EXPENSIVE TO CHANGE LATER

Once 5,000 KUIDs exist with no identity anchor, deduplicating them is a manual programme measured in months. The unique constraint costs one line of SQL today.

#### 3 — THE VERIFICATION STATE MACHINE CANNOT UNDO AN APPROVAL

`draft → payment_pending → under_review → approved | rejected` has no exit from `approved`. But verification *is* the product. The first time you approve someone on a forged document — and in a 5,000-person pilot with cash-handling agents there will be a first time — you will need to withdraw that badge and there will be no code path to do it.

Add `approved → revoked`: super\_admin only, mandatory reason, audit row, no refund, and the public profile shows an explicit "verification withdrawn" state rather than silently reverting to unverified. Add it now — adding a state to a live state machine is a migration plus a UI pass plus a policy argument.

#### 4 — "RESUBMIT AGAINST THE SAME PAYMENT\_ID" IS AN UNBOUNDED LOOP

The rule is right: a rejected athlete should not pay twice. But as written it is an infinite free-review coupon and your reviewers absorb the cost. Cap it at **three submissions per payment**, after which the case escalates to a coordinator who resolves it in person or refunds. Store the count on the verification row.

#### 5 — 72-HOUR EXPIRY CAN RACE A LATE WEBHOOK, AND MONEY MUST WIN

If `payment_pending` expires to `draft` after 72 hours, what happens when Paystack's webhook lands at hour 73 — a retry after an outage — reporting a successful charge? As specified, you have taken ₦2,500 and the record says `draft`.

State the rule in one line and enforce it in code: **a confirmed payment always wins, whatever state the record is in**. Expiry may only touch records with no successful charge; a late success moves the record to `under_review` from wherever it sits.

## 6 — THERE IS NO REFUND PATH, AND YOU WILL NEED ONE

"No withdrawal endpoint" is correct and I am not reopening it. But chargebacks and disputes are not withdrawals — they are Paystack reversing a charge, and they happen whether or not you designed for them. Today the ledger cannot represent one, so your books will drift from your bank statement the first time it occurs.

The resolution keeps you well clear of the CBN: **refunds are initiated by a human inside Paystack's own dashboard, never by your application.** Your system only *records* the reversal, as a debit ledger line created by a super\_admin action with a mandatory reason. No endpoint moves money; one endpoint records that money moved elsewhere. You remain a record book.

## 7 — PAYSTACK'S FEE IS MISSING FROM THE LEDGER DESIGN

You charge ₦2,500. Paystack takes roughly 1.5% + ₦100 on local cards, so about ₦2,362 reaches your account. If the ledger records only ₦2,500, **your books will never reconcile against your bank settlement** — and you will find out in week 6, when someone asks how much the pilot actually made.

Record gross and fee as two separate ledger lines from the very first payment. And decide today whether ₦2,500 is the price or the take: if the CEO expects ₦2,500 net per athlete, the price is ₦2,640.

## 8 — OPERA MINI WILL BREAK A JAVASCRIPT-HEAVY APP, AND IT IS POPULAR IN THE NORTH

"Works on any Android browser" hides a hard constraint. Opera Mini in its default proxy mode renders pages on Opera's own servers with severely limited JavaScript and **no service worker at all**. UC Browser behaves similarly. On entry-level Android over 2G/3G in northern Nigeria, these are not edge cases.

### ARCHITECTURAL CONSTRAINT, NOT A PREFERENCE

Every critical flow — registration, login, starting a payment, viewing a public profile — must work as **server-rendered HTML with plain form POSTs**. JavaScript is enhancement, never a dependency. The PWA install and offline shell are a bonus for capable browsers, not the delivery mechanism. Budget: first-load JavaScript under 150KB gzipped.

Cheap if it is a rule from day one. A rewrite if it is discovered in week 6.

## 9 — THE PAYWALL HAS A HOLE: R2 OBJECT URLS

Photos are the thing people pay ₦2,500 for. If uploaded objects sit in a public R2 bucket, anyone who obtains or guesses an object URL bypasses the paywall, and one leaked URL pattern defeats the business model.

- The bucket is **private**. Uploads still go direct to R2 via presigned PUT — that part of the spec is right, keep it.
- Reads go through short-lived presigned GET URLs issued by the app *after* a permission check, or through a signed image path on your own domain. Either way the app decides, every time.
- Uploaded files are untrusted input. The worker re-encodes every image into a derivative and strips EXIF (which carries GPS coordinates). The derivative is what gets served; the original is never public.

## 10 — THE QR TICK MEANS LESS THAN PEOPLE WILL ASSUME

The signature is `HMAC-SHA256(kuid, QR_SECRET)` — deterministic per KUID, embedded in a URL printed on a card. Anyone who scans one card can copy the whole URL, signature included. So the tick proves "**this QR was generated by KAFRIADA.**" It does not prove the bearer is the person on screen, and it does not prove the card was not photocopied.

Still worth having — but say so plainly on the page ("Issued by KAFRIADA. Check the photo matches the bearer.") rather than letting a scout believe the tick means more. Two cheap additions now:

- **Put a key version in the signature:** `?s=1.a1b2c3...`. If `QR_SECRET` leaks you rotate to key 2 and re-issue, instead of invalidating every printed card in Jigawa at once.
- Make sure the Cloudflare cache key includes the query string, or you will cache a ticked page and serve it to an unsigned request.

### EXPENSIVE TO CHANGE LATER

The key version costs two characters in a string today. Retrofitting it makes every card already printed unverifiable the day you rotate.

## 11 — ASSISTED PAYMENT IS AN OPERATIONS RISK WEARING AN ARCHITECTURE COSTUME

An LGA coordinator takes cash and pays with their own card. Three failure modes, none of them code bugs:

- **The coordinator keeps the cash and never pays.** The athlete has no receipt and no recourse.
- **The coordinator's card is charged 27 times in a day** and their bank or Paystack's fraud engine blocks it mid-drive.
- **The coordinator disputes their own charges** afterwards, and you have given away 27 verifications.

Architecture cannot fix dishonesty, but it must make it visible and bounded: an SMS receipt to the *athlete's* phone the moment the webhook confirms (KUID, amount, timestamp, coordinator name); hard per-coordinator daily caps on both count and naira; and a settlement report operations reconciles against cash handed in. Design in §4c.

## 12 — A STALLED TRANSFER HAS NOWHERE TO SIT

Requiring the buying club to hold an approved org account before confirming is a deliberate growth mechanism and I like it. But a transfer can then sit in `player_accepted` for weeks while a club registers and pays ₦15,000 — and to everyone watching it simply looks broken. Give it a visible state (`awaiting_buyer_onboarding`), a nudge to the buying club's contact, and an expiry so dead transfers close themselves instead of accumulating.

## 13 — NOBODY IS NAMED AS THE VERIFIER, AND SELF-APPROVAL IS POSSIBLE

The state machine has `under_review → approved` but never says who holds that power or what stops abuse. At pilot volume — roughly 200 paid verifications over eight weeks, about four a day — this is trivially manual. But the rule must exist before the first approval, not after the first scandal: **a reviewer may not approve their own athlete record, nor an athlete of a club they administer.** Enforced in code, not in a policy document.

## 14 — "AUDIT LOGS CANNOT BE DELETED" IS NOT TRUE IF THE APP OWNS THE TABLE

If the application connects to Postgres as the table owner, the only thing preventing deletion of an audit row is your own code — and your own code is exactly what an attacker who reaches the app gets to run. Enforce it where it is actually enforceable: the app connects as a restricted role holding `INSERT` and `SELECT` on `audit_logs` and no `UPDATE` or `DELETE`, granted by migration. Two lines of SQL — the difference between a claim and a control.

## 15 — SMALLER THINGS, A LINE EACH

- The KUID encodes the LGA of *registration*. People move. State publicly that it is a birth-certificate number, not an address, or you will spend the pilot refusing re-issue requests from coordinators who read it as current location.
- Uniqueness is `(state, year, serial)`; the LGA segment is informational. Write that down so nobody "fixes" the counter key later.
- Session lifetime is unspecified. Coordinators work on shared phones — staff roles need a 30-minute idle timeout; athletes can have 30 days.
- "500 concurrent users" against 5,000 total athletes is an arbitrary safety factor, not a modelled load. The real shape is *bursty*: 200 registrations in an hour at a rally, then a spike of QR scans on match day. Design for the burst.
- 18+ is self-declared at registration; only Stage-2 review puts human eyes on a document. So 96% of records are never age-checked. Accept that consciously rather than by accident.

## B. Assumptions you appear to be making but have not stated

---

### THAT YOU WILL NEVER REFUND

You will. See A.6.

---

### THAT A PHONE NUMBER IDENTIFIES A PERSON

It mostly does in practice, but nothing in the spec commits to it. See A.2.

---

### THAT COORDINATORS ARE TRUSTWORTHY

Most are. The design has to survive the ones who are not. See A.11.

---

### THAT EVERYONE HAS A MODERN BROWSER

See A.8 — this one changes how you write pages.

---

### THAT ₦2,500 IS WHAT YOU RECEIVE

It is what you charge. See A.7.

---

THAT R2 IS COVERED BY YOUR BACKUPS

Object storage is not inside a Postgres snapshot. See §5.

THAT A COUNTER ROW AND A SEQUENCE ARE EQUIVALENT

They are not. One gaps and does not lock; the other locks and does not gap. See §4a.

THAT 5,000 SUBJECTS IS BELOW THE NDPR THRESHOLD

It is not — filing obligations begin above 2,000 subjects in twelve months. See §6.

C. The three decisions most expensive to change after week 8

| # | DECISION                                          | WHY IT CANNOT BE UNDONE                                                                                                                                                                                                                                                                               | SETTLE IT THIS WEEK BY                                                                                                                                                                                       |
|---|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | What a KUID means, and what anchors it to a human | Every printed card, poster, QR sticker and club record in Jigawa carries it. It is immutable by design, so there is no migration — only a re-issue programme, in person, across 27 LGAs. Duplicates found later cannot be merged without breaking the immutability promise.                           | Unique phone constraint in the schema. A published statement that the KUID records place of registration, not residence. A key version in the QR signature.                                                  |
| 2 | The shape of the wallet ledger                    | Once real money from real people is in it, rewriting the ledger means re-deriving history and then defending that derivation to people, about their own money. Gross-versus-net, immutability, and whether balance is stored or derived are all one-way doors.                                        | Insert-only <code>ledger_entries</code> ; integer kobo; gross and fee as separate lines; balance derived, with a cached column reconciled nightly.                                                           |
| 3 | The permission model, including scope             | Not the roles — those are table rows and cheap. The expensive part is <i>scope</i> : that a <code>club_admin</code> 's authority stops at one club and a coordinator's at one LGA. Bolt scope on later and every route must be re-audited; until it is, the system quietly leaks other people's data. | <code>user_roles</code> carries <code>scope_type</code> and <code>scope_id</code> from the first migration. Middleware checks permission <i>and</i> scope. CI fails any route without a declared permission. |

*Runner-up*: the no-JavaScript-required rule (A.8). Not irreversible, but retrofitting it costs a full UI pass.

## 02 Requirements

What it must do, kept separate from how well it must do it. Every non-functional requirement carries a number you can test against.

### Functional requirements

| REF    | AREA              | THE SYSTEM MUST...                                                                                                                                                                                           |
|--------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F-ID-1 | Identity          | Register an adult athlete with name, verified phone, date of birth, LGA, sport and position, at no cost.                                                                                                     |
| F-ID-2 |                   | Issue exactly one immutable KUID per athlete, in the same transaction that creates the record, and never reuse or re-issue one.                                                                              |
| F-ID-3 |                   | Verify the phone by one-time SMS code before the record becomes usable.                                                                                                                                      |
| F-ID-4 |                   | Publish a public profile at <code>/a/{kuid}</code> readable with no account, showing a silhouette rather than a photo for unverified athletes.                                                               |
| F-ID-5 |                   | Generate a signed QR image encoding that URL, downloadable and printable by the athlete and by coordinators in bulk.                                                                                         |
| F-ID-6 |                   | Maintain an append-only career history — club joins, departures, completed transfers — visible on the public profile.                                                                                        |
| F-VF-1 | Verification      | Accept a Stage-2 submission with an identity document and a photograph, priced ₦2,500 for athletes and ₦15,000 for clubs.                                                                                    |
| F-VF-2 |                   | Move a record through <code>draft → payment_pending → under_review → approved   rejected</code> , plus <code>revoked</code> from approved.                                                                   |
| F-VF-3 |                   | Allow up to three resubmissions against one payment after rejection, then escalate.                                                                                                                          |
| F-VF-4 |                   | Expire an unpaid <code>payment_pending</code> back to <code>draft</code> after 72 hours — but never a record with a confirmed charge.                                                                        |
| F-VF-5 |                   | Prevent a reviewer approving their own record or an athlete of a club they administer.                                                                                                                       |
| F-PY-1 | Payments & wallet | Take money only through Paystack, and treat the webhook — not the browser redirect — as the truth about success.                                                                                             |
| F-PY-2 |                   | Process every webhook idempotently: a replay must change nothing and still return 200.                                                                                                                       |
| F-PY-3 |                   | Record every movement as insert-only ledger lines in integer kobo, gross and provider fee separately.                                                                                                        |
| F-PY-4 |                   | Let an LGA coordinator pay on an athlete's behalf, with the ledger entries landing on the athlete and the coordinator recorded on the payment.                                                               |
| F-PY-5 |                   | Expose <i>no</i> withdrawal and <i>no</i> peer-to-peer endpoint. Not disabled — absent.                                                                                                                      |
| F-CL-1 | Clubs             | Let a club register free, hold a roster of athletes by KUID, and appoint club administrators.                                                                                                                |
| F-CL-2 |                   | Require an athlete to accept a roster invitation before appearing on it.                                                                                                                                     |
| F-CL-3 |                   | Track a club's org-account status, which gates the ability to confirm an incoming transfer.                                                                                                                  |
| F-TR-1 | Transfers         | Record a transfer through <code>listed → player_accepted → awaiting_buyer_onboarding → buyer_confirmed → validated → completed</code> , with <code>cancelled</code> reachable from any pre-completion state. |
| F-TR-2 |                   | Move no money and hold no transfer fee at any point.                                                                                                                                                         |
| F-TR-3 |                   | On completion, close the old roster membership, open the new one, write a career event and publish a news post targeted at both LGAs — all atomically.                                                       |
| F-FD-1 | Feed              | Let an admin publish a post targeted at the whole state, named LGAs, or everyone.                                                                                                                            |
| F-FD-2 |                   | Show each user the posts targeted at their LGA or state, newest first. Read-only for everyone but admins.                                                                                                    |
| F-AD-1 | Admin & core      | Enforce table-driven role permissions, scoped to a club or an LGA where the role is scoped, on every route, denying by default.                                                                              |
| F-AD-2 |                   | Write an insert-only audit row for every state change, money movement, permission change and admin action.                                                                                                   |
| F-AD-3 |                   | Give coordinators a review queue, a settlement report, and per-LGA registration statistics.                                                                                                                  |
| F-AD-4 |                   | Support manual data export and anonymisation on request, recorded in the audit log.                                                                                                                          |

### Non-functional requirements

Adjectives are unfalsifiable, so every line below is a number the load test or a dashboard can check. Where the Build Spec gave a target I have kept it and said what it excludes.

| REF       | QUALITY         | MEASURABLE TARGET                                                                                                       | HOW IT IS VERIFIED                                                                          |
|-----------|-----------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| N - P - 1 | Performance     | Authenticated API p95 < 500ms, p99 < 1,200ms, measured at the origin, excluding time spent inside Paystack.             | k6 load test in staging; Sentry performance in production.                                  |
| N - P - 2 |                 | Public profile p95 < 200ms as measured from Lagos, with an edge cache hit ratio above 85%.                              | Cloudflare analytics; synthetic check from a Lagos probe.                                   |
| N - P - 3 |                 | KUID mint contention p99 < 5ms; 200 registrations inside 60 seconds with zero errors and zero duplicate serials.        | Dedicated burst test, asserted on <code>COUNT(DISTINCT kuid) = COUNT(*)</code> .            |
| N - P - 4 |                 | 500 concurrent virtual users, mixed 70% profile view / 20% browse / 8% register / 2% pay, error rate below 0.5%.        | k6, run once per week from week 4.                                                          |
| N - P - 5 |                 | First-load JavaScript ≤ 150KB gzipped; LCP < 2.5s on a throttled Moto-G-class device over slow 3G.                      | Lighthouse in CI, failing the build on regression.                                          |
| N - A - 1 | Availability    | 99.5% monthly uptime — about 3.6 hours of downtime allowed per month.                                                   | External uptime monitor. <i>This is an honest single-host number; do not promise 99.9%.</i> |
| N - A - 2 |                 | Registration and public profiles stay available when Paystack, Termii or R2 is down.                                    | Dependency-failure drill in staging, see §5.                                                |
| N - A - 3 |                 | Planned deploys cause zero dropped requests.                                                                            | Rolling deploy with a health check; verified under load.                                    |
| N - M - 1 | Money integrity | Zero lost webhooks: every webhook is persisted before it is processed.                                                  | Row count in <code>webhook_events</code> reconciled against Paystack's dashboard daily.     |
| N - M - 2 |                 | Webhook to ledger-committed in under 5 seconds, 99.9% of the time.                                                      | Timestamp delta on <code>webhook_events</code> , alerted.                                   |
| N - M - 3 |                 | No payment left unresolved for more than 25 hours.                                                                      | Hourly reconciliation job; the count is a dashboard tile and pages on rise.                 |
| N - M - 4 |                 | For every wallet, sum of credits minus sum of debits equals the cached balance. Zero exceptions, checked nightly.       | Nightly integrity job; any mismatch pages immediately.                                      |
| N - R - 1 | Recoverability  | RPO ≤ 5 minutes via point-in-time recovery; RTO ≤ 4 hours.                                                              | Proven by a rehearsed restore before launch, not by the provider's brochure.                |
| N - R - 2 |                 | Daily snapshot, 30-day retention, plus R2 object versioning with a 30-day non-current retention.                        | Backup-success alert; a failed backup opens a ticket the same morning.                      |
| N - R - 3 |                 | Audit rows retained 7 years and never mutated.                                                                          | Database grants; a migration test asserting the app role cannot delete.                     |
| N - S - 1 | Security        | Every API route declares a required permission; a route without one fails the build.                                    | CI route-manifest test.                                                                     |
| N - S - 2 |                 | All 8 roles asserted against all routes — roughly 500 generated assertions — on every pull request.                     | Generated permission-matrix test suite.                                                     |
| N - S - 3 |                 | Rate limits: OTP 1 per 60s and 5 per day per phone; login 10/hour per account and per IP; payment init 10/day per user. | Integration tests plus Cloudflare rules.                                                    |
| N - O - 1 | Operability     | Verification decided within 24 hours for 95% of paid submissions during the pilot.                                      | Queue-age dashboard; oldest-item age alerts at 18 hours.                                    |
| N - O - 2 |                 | Every request carries a tracking ID, surfaced on error screens so a coordinator can read it down the phone.             | Manual check plus a log-correlation test.                                                   |

### Scale: today's numbers, and the path to Year 1

Worth stating plainly, because it justifies most of the "do not build that" decisions later in this document.

| DIMENSION          | PILOT (8 WEEKS) | YEAR 1 TARGET | WHAT CHANGES BETWEEN THEM                                                           |
|--------------------|-----------------|---------------|-------------------------------------------------------------------------------------|
| Athletes           | 5,000           | 10,000        | Nothing. A 10,000-row table is small.                                               |
| Clubs              | 81              | 300+          | Nothing.                                                                            |
| States / LGAs      | 1 / 27          | 6–10 / ~200   | The KUID counter stops being a single hot row — contention falls, it does not rise. |
| Database size      | ~60 MB          | ~250 MB       | Fits in RAM in both cases. No sharding, no read replica, no cache tier.             |
| Media in R2        | ~1.5 GB         | ~4 GB         | Well inside the free tier. Cost is dominated by nothing.                            |
| Paid verifications | ~200            | ~1,600        | 4/day becomes ~6/day. Manual review still works; add reviewers, not software.       |
| Peak write burst   | 200 / minute    | 400 / minute  | Still an order of magnitude below the counter's ceiling (§4a).                      |

#### WHAT THIS TABLE IS FOR

The honest conclusion is that **a single application instance and a single Postgres will carry you comfortably past the Year-1 target**. The scaling work in this document is therefore about *bursts* and *failure*, not about volume. Anything proposed on volume grounds before 10,000 athletes should be refused, and this table is the refusal.

## 03 Module map

One job per module, and one rule between them: a module may call another module's exported functions, and may never read another module's tables.

The rule matters more than the folders. It is what makes it possible to lift `wallet` or `identity` out into its own service in year two without archaeology — and it is worth exactly nothing unless it is enforced mechanically. So:

HOW THE BOUNDARY IS ENFORCED, NOT MERELY STATED

- Each module exports one `service.ts`. Everything else in the module is private.
- An ESLint `no-restricted-imports` rule fails the build on any import that reaches past a module's `service.ts` into its internals or its schema.
- Cross-module database work that must be atomic passes a transaction handle into the service function. Services accept an optional `tx` — this is the one concession, and without it \$4d's transfer completion cannot be atomic.

### The 23 tables

The Build Spec said "roughly 18". The honest count with everything this document requires is 23. The five extra are `webhook_events`, `transfer_events`, `notifications_outbox`, `consents` and `rate_limits` — every one of them exists because of a specific failure discussed in \$1 or \$5, and I would not cut any of them. If you must cut, `transfer_events` can fold into the audit log and `post_targets` into an array column, saving two tables and costing you queryability.

|                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| core                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| core/auth · core/rbac · core/locations · core/notifications · core/audit · core/media |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| RESPONSIBILITY                                                                        | Everything every module needs and no module should reimplement: who you are, what you may do, where places are, how messages get sent, what happened, and where files live. Core knows nothing about athletes, clubs, transfers or money.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| OWNS                                                                                  | <code>users</code> , <code>sessions</code> , <code>roles</code> , <code>role_permissions</code> , <code>user_roles</code> , <code>locations</code> , <code>audit_logs</code> , <code>notifications_outbox</code> , <code>media_assets</code> , <code>consents</code> , <code>rate_limits</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| EXPORTS                                                                               | <ul style="list-style-type: none"><li>• <code>auth.createUser(phone, passwordHash, tx?)</code> · <code>auth.verifySession(cookie)</code> · <code>auth.requireReauth(userId)</code></li><li>• <code>rbac.can(userId, permission, scope?)</code> · <code>rbac.grant(userId, role, scope, actorId, tx)</code></li><li>• <code>locations.resolveLga(id)</code> · <code>locations.isLive(lgaId)</code> · <code>locations.stateCodeFor(lgaId)</code></li><li>• <code>notify.enqueue(channel, target, template, vars, dedupeKey, tx?)</code></li><li>• <code>audit.write(actorId, action, subjectType, subjectId, meta, tx)</code></li><li>• <code>media.presignUpload(ownerType, ownerId, kind)</code> · <code>media.readUrl(assetId, viewerId)</code></li></ul> |
| NEVER TOUCHES                                                                         | Any module table. Core is a leaf — nothing it does may depend on a business module, or you get an import cycle on day three.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| RIPPLE RISK                                                                           | Everything depends on <code>rbac.can</code> and <code>audit.write</code> , so a change to either signature touches the whole codebase. Freeze both signatures in week 1 and treat them as public API. Add optional parameters, never change existing ones.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

## identity

modules/identity

### RESPONSIBILITY

An athlete's record, their KUID, their verification state, and their career history. The single authority on whether an athlete is verified — nothing else in the system may decide that.

### OWNS

`athletes`, `kuid_counters`, `verifications`, `career_events`

### EXPORTS

- `identity.register(input)` — the whole Stage-1 flow including the KUID mint
- `identity.getPublicProfile(kuid)` — the projection the public page renders, with photo suppressed unless approved
- `identity.summary(athleteId)` — the small shape other modules embed (kuid, name, lga, verified)
- `identity.markPaid(verificationId, paymentId, tx)` — called only by wallet on webhook confirmation
- `identity.decide(verificationId, reviewerId, outcome, reason, tx)`
- `identity.recordCareerEvent(athleteId, type, clubId, meta, tx)` — called by transfers and clubs

### NEVER TOUCHES

`payments`, `ledger_entries`, `clubs`, `club_members`, `transfers`. It may not ask "has this athlete paid" by reading a payments row — wallet tells it, once, by calling `markPaid`.

### RIPPLE RISK

Every module wants to render an athlete's name and verified badge. If they each query `athletes`, a schema change breaks five modules at once. **Prevention:** `identity.summary()` is the only permitted shape, and the ESLint rule makes the alternative impossible.

## wallet

modules/wallet

### RESPONSIBILITY

Every naira the system has ever seen, and the only code that talks to Paystack. It is a record book: it records that money arrived, and has no vocabulary for money leaving except a human-recorded reversal.

### OWNS

`wallets`, `ledger_entries`, `payments`, `webhook_events`

### EXPORTS

- `wallet.initiate(purpose, beneficiaryAthleteId, payerUserId, coordinatorId?)` → authorization URL
- `wallet.handleWebhook(rawBody, signature)` → the whole idempotent path of \$4b
- `wallet.reconcile(sinceHours)` — the worker's hourly job
- `wallet.recordReversal(paymentId, actorId, reason)` — super\_admin only, records a refund made by hand in Paystack
- `wallet.statement(walletId)` · `wallet.coordinatorSettlement(coordinatorId, dateRange)`

### NEVER TOUCHES

`athletes`, `verifications`, `clubs`. On a confirmed payment it calls `identity.markPaid()` or `clubs.markPaid()` inside its own transaction. It does not know what verification *means* — only that something was paid for.

### NEVER EXISTS

No `withdraw`, no `transferTo`, no `payout`. Not commented out, not feature-flagged, not present. A CI grep for those identifiers fails the build — cheap, and it survives staff turnover better than a paragraph in a wiki.

### RIPPLE RISK

The temptation is to let wallet set `athletes.verification_state` directly because it is one line. That single line is how the identity state machine ends up with two authors and an unexplainable state six weeks later. **Prevention:** table grants — the app's wallet code paths and identity's tables are separated by the ESLint rule, and the review checklist names this specific shortcut.

## clubs

modules/clubs

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RESPONSIBILITY | A club, who administers it, who plays for it now, and whether it holds an approved org account.                                                                                                                                                                                                                                                                                                                                                   |
| OWNS           | <code>clubs</code> , <code>club_members</code>                                                                                                                                                                                                                                                                                                                                                                                                    |
| EXPORTS        | <ul style="list-style-type: none"><li><code>clubs.summary(clubId)</code> · <code>clubs.isApprovedOrg(clubId)</code> — the gate transfers depends on</li><li><code>clubs.roster(clubId)</code> · <code>clubs.currentClubOf(athleteId)</code></li><li><code>clubs.openMembership(clubId, athleteId, tx)</code> · <code>clubs.closeMembership(clubId, athleteId, at, tx)</code></li><li><code>clubs.markPaid(clubId, paymentId, tx)</code></li></ul> |
| NEVER TOUCHES  | <code>athletes</code> (uses <code>identity.summary</code> ), <code>transfers</code> , <code>payments</code> .                                                                                                                                                                                                                                                                                                                                     |
| RIPPLE RISK    | "Which club is this athlete at" is asked by identity, transfers and feed. If any of them derive it by reading <code>club_members</code> , the rule that a membership is closed before the next opens becomes three people's responsibility. <b>Prevention:</b> <code>currentClubOf()</code> is the only answer, and the invariant — at most one open membership per athlete — is a partial unique index in the database, not a code convention.   |

## transfers

modules/transfers

|                |                                                                                                                                                                                                                                                                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RESPONSIBILITY | The transfer state machine and its evidence trail. It moves no money, ever — that is the point of the design, not a limitation of it.                                                                                                                                                                                                 |
| OWNS           | <code>transfers</code> , <code>transfer_events</code>                                                                                                                                                                                                                                                                                 |
| EXPORTS        | <ul style="list-style-type: none"><li><code>transfers.list(...)</code> · <code>transfers.accept(...)</code> · <code>transfers.confirmBuyer(...)</code> · <code>transfers.validate(...)</code> · <code>transfers.cancel(...)</code></li><li><code>transfers.historyFor(athleteId)</code> — read model for the public profile</li></ul> |
| NEVER TOUCHES  | <code>club_members</code> , <code>athletes</code> , <code>career_events</code> , <code>posts</code> . Completion calls into clubs, identity and feed — all inside one transaction it owns.                                                                                                                                            |
| RIPPLE RISK    | The highest-fan-out operation in the system: completing a transfer touches four modules. <b>Prevention:</b> one function, <code>completeTransfer(tx)</code> , is the only place that orchestration exists, and it is the subject of a dedicated integration test that asserts either all five effects happened or none did.           |

## feed

modules/feed

|                |                                                                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RESPONSIBILITY | Admin-authored posts, targeted at a state or named LGAs. Read-only for everyone else.                                                                                     |
| OWNS           | <code>posts</code> , <code>post_targets</code>                                                                                                                            |
| EXPORTS        | <code>feed.publish(authorId, content, targets, tx?)</code> · <code>feed.forUser(userId, cursor)</code> · <code>feed.publishSystemPost(template, vars, targets, tx)</code> |
| NEVER TOUCHES  | Anything. Feed is the most isolated module in the system and should stay that way.                                                                                        |
| RIPPLE RISK    | Almost none — which is why the automatic transfer announcement lives here behind <code>publishSystemPost</code> rather than transfers writing a post row itself.          |

|                               |                                                                                                                                                                                                                                                                                                                     |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>admin</b><br>modules/admin |                                                                                                                                                                                                                                                                                                                     |
| RESPONSIBILITY                | Screens and read models for coordinators and super_admins: the review queue, LGA statistics, the settlement report, user management, the audit viewer. <b>It owns no tables and holds no business rules.</b>                                                                                                        |
| OWNS                          | Nothing. Deliberately.                                                                                                                                                                                                                                                                                              |
| EXPORTS                       | Nothing to other modules. Admin is a consumer, never a dependency.                                                                                                                                                                                                                                                  |
| RIPPLE RISK                   | Admin is where rules go to get duplicated, because an admin screen always needs one thing the service does not quite return. <b>Prevention:</b> admin may only call exported services. When it needs a new shape, the answer is a new exported read function on the owning module — never a query written in admin. |

**The dependency graph, and the one rule it obeys**

Dependencies point in one direction. Nothing points back into core, and nothing points into `admin` at all.

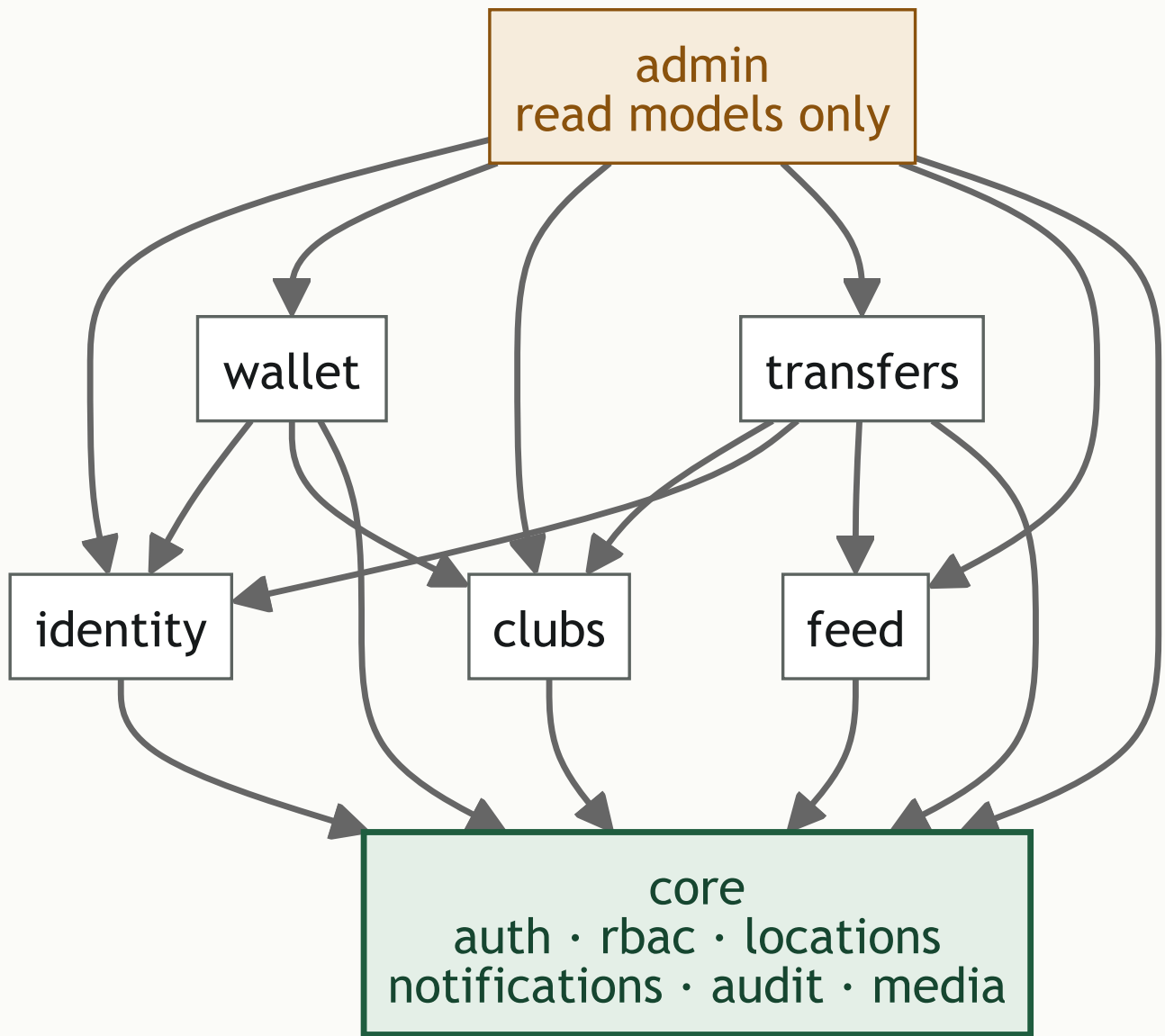


FIGURE 1 — MODULE DEPENDENCY GRAPH. ARROWS ARE "CALLS THE EXPORTED SERVICE OF". NO ARROW IS BIDIRECTIONAL; NO CYCLE EXISTS.

## Where a change ripples, and the mechanism that stops it

| CHANGE HERE...                    | ...COULD BREAK THIS                                  | WHAT ACTUALLY PREVENTS IT                                                                                                                                                                                                       |
|-----------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>athletes</code> schema      | Every module rendering a name or badge               | <code>identity.summary()</code> is the only shape allowed out. The ESLint import rule makes the shortcut a build failure, not a code-review conversation.                                                                       |
| Verification state machine        | wallet (on payment), admin (queue), public profile   | Transitions are only reachable through <code>identity.decide()</code> and <code>identity.markPaid()</code> . A table-driven transition map rejects any move not in it, so an unlisted transition throws rather than corrupting. |
| Ledger entry shape                | Statements, settlement report, nightly integrity job | <code>ledger_entries</code> is insert-only and additive-only: new <code>kind</code> values, never changed meanings. The nightly balance check is the canary.                                                                    |
| <code>rbac.can()</code> signature | Every route in the system                            | Frozen in week 1. Scope is a parameter from the first migration precisely so it never needs adding later (§1.C.3).                                                                                                              |
| Transfer completion               | clubs, identity, feed — atomically                   | One orchestration function, one transaction, one integration test asserting all-or-nothing.                                                                                                                                     |
| Adding a role                     | Nothing                                              | Roles and permissions are table rows. The GDOC's other 20 roles are future <code>INSERT</code> s, not future code — that was the point of table-driven RBAC.                                                                    |

## 04 The critical flows

Five flows, end to end, with the concurrency and idempotency arguments written out rather than asserted.

### a) Stage-1 registration and the atomic KUID mint

The whole flow, in order, with the transaction boundary marked. Everything expensive happens outside it.

- 1 **Browser** → **Cloudflare** → **app**. A plain HTML form POST. No JavaScript required (§1.A.8).
- 2 **Validate and normalise**. Phone to E.164, date of birth checked for 18+, LGA checked against `locations` and against `is_live` — registration is closed in LGAs not yet rolled out (§7).
- 3 **Rate limit**. Per IP and per phone, in Postgres. Rejected here costs nothing downstream.
- 4 **Hash the password with argon2id**. **OUTSIDE THE TRANSACTION** 100–300ms of CPU. This is the single most important ordering decision in the flow.
- 5 **BEGIN**. The transaction opens here and is designed to last under 8ms.
- 6 `INSERT INTO users` — unique index on `phone`. A duplicate raises 23505 and we roll back having minted nothing.
- 7 `INSERT INTO athletes` with a null `kuid`, returning the id.
- 8 **Mint**. One statement, shown below. This is the last write that can block, and it is the second-to-last statement in the transaction.
- 9 `UPDATE athletes SET kuid = ...` and `INSERT INTO audit_logs`.
- 10 **COMMIT**. The counter lock is released here — roughly 2ms after step 8 acquired it.
- 1 **After commit**: enqueue the OTP SMS through `notify.enqueue`. Never send an SMS inside a transaction — Termii being slow must not hold a database lock.
- 2 **Respond** with the KUID and a redirect to phone verification.

```
-- The entire mint. One statement, one row lock, no read-modify-write.
INSERT INTO kuid_counters (year, state_code, next_serial)
VALUES ($1, $2, 1)
ON CONFLICT (year, state_code)
DO UPDATE SET next_serial = kuid_counters.next_serial + 1
RETURNING next_serial;

-- KUID assembled in application code, never in SQL:
-- KA-NG-JG-BKD-2026-000123
-- | | | | | serial, zero-padded to 6
-- | | | | year of registration
-- | | | LGA of registration (informational only)
-- | | state (part of the uniqueness key)
-- | country
-- KAFRIADA
--
-- Uniqueness is (state, year, serial). The LGA segment is decorative.
-- Do not change the counter key later without reading §1.A.15.
```

### THE CONCURRENCY PROOF: 200 PEOPLE REGISTER IN THE SAME SECOND

The question is only ever "how long is the counter row locked, and by what."

- The lock is taken by the `UPDATE` in step 8 and released at `COMMIT` in step 10. Between them sit one `UPDATE` and one `INSERT` against small, indexed, uncontended tables.
- Measured hold time is therefore **1–3ms**, dominated by network round trips inside the same availability zone, not by computation.
- Postgres serialises the waiters on that row. Throughput ceiling  $\approx 1 \div \text{hold time} \approx \mathbf{330\text{--}1,000 \text{ mints per second}}$ .
- 200 registrations arriving in one second queue at most  $200 \times 3\text{ms} = \mathbf{600\text{ms of aggregate wait}}$ , with the unluckiest request waiting that long and the median waiting  $\sim 300\text{ms}$ . Inside the p95 budget, with headroom.

- Compare the spec as written, with argon2id inside the transaction: hold time 250ms, ceiling 4/second, and those 200 registrations take 50 seconds with most of them timing out. **That one ordering decision is the difference between working and not.**

#### WHY A COUNTER ROW AND NOT A POSTGRES SEQUENCE

A sequence would remove contention entirely — it is non-transactional and never blocks. It also *leaves gaps*: a rolled-back registration burns its number permanently. That is technically harmless and operationally poisonous. A coordinator holding cards 000121 and 000123 will ask where 000122 went, and "it does not exist" is not an answer that builds trust in an identity register in its first month.

So: **counter row, gapless, ~500/s ceiling**. The trade is contention we have measured against a question we would have to answer forever. If you ever exceed the ceiling, the counter is per `(year, state)` — adding states divides the load automatically.

#### WHAT HAPPENS WHEN THE REQUEST IS RETRIED

The three cases that matter, and why none of them mints twice:

- **Retry before commit** (client timeout, user double-taps Submit) — the second request's `INSERT INTO users` either blocks on the unique index and then fails with `23505`, or fails immediately. Roll back; nothing minted.
- **Retry after commit but before the response arrived** — same `23505`. We catch it specifically, look up the existing athlete by phone, and return *their existing KUID* with 200. The user sees success, which is true.
- **Crash between commit and response** — indistinguishable from the above, resolved the same way.

#### THE INVARIANT, STATED SO IT CAN BE TESTED

A KUID cannot exist without its athlete row, because both are written in one transaction. An athlete row cannot exist without a KUID, because the same transaction fills it before committing. And one phone number can never produce two KUIDs, because the database index says so and not because the code remembers to check. The burst test asserts `COUNT(*) = COUNT(DISTINCT kuid)` after 200 concurrent registrations including deliberate duplicates.

#### b) Stage-2 payment, from initialise to approval

Money is the part where "mostly works" is not a grade. Two things carry this flow: the webhook is the truth, and the webhook is idempotent.

#### PHASE 1 — INITIALISE

- 1 Athlete submits documents and photo. Files go **direct to R2** via presigned PUT; only the object keys reach the app.
- 2 App creates a `verifications` row in `draft` and a `payments` row in `initialized` with **our own reference**, `KAF-{uuid}`. We generate the reference so we can always find our record from Paystack's payload and vice versa.
- 3 App calls Paystack `/transaction/initialize` with that reference, the amount in kobo, and the callback URL.
- 4 Verification moves to `payment_pending`; the browser is redirected to Paystack's page. **We take no card details, ever.**

#### PHASE 2 — THE WEBHOOK, AND EXACTLY HOW A REPLAY IS MADE HARMLESS

Paystack retries webhooks. Networks duplicate them. Assume every webhook arrives at least twice and possibly out of order.

```

POST /api/webhooks/paystack      runtime = 'nodejs'   CSRF: exempt

1  const raw = await req.text()      // RAW body. Never req.json() first –
                                     // re-serialising changes byte order and
                                     // the HMAC will never match again.
2  verify  HMAC-SHA512(raw, PAYSTACK_SECRET) === header['x-paystack-signature']
    └ mismatch → 401, log, alert if it repeats. No database work.

3  INSERT INTO webhook_events (provider, provider_event_id, payload)
   VALUES ('paystack', $eventKey, $raw)
   ON CONFLICT (provider, provider_event_id) DO NOTHING
   RETURNING id;
    └ no row returned → WE HAVE SEEN THIS BEFORE → return 200, do nothing.
      This single statement is the whole replay defence. It is atomic,
      it is enforced by a unique index, and it works across concurrent
      deliveries of the same event because the index arbitrates, not the code.
      $eventKey = data.reference + ':' + event

4  BEGIN
5  SELECT * FROM payments WHERE reference = $ref FOR UPDATE
6  guard: payment.status === 'success' → COMMIT, return 200   (second belt)
7  guard: data.status === 'success'
      AND data.currency === 'NGN'
      AND data.amount === payment.amount_kobo   ← never skip this one
      └ amount mismatch → freeze, flag for human, alert. Do NOT approve.
8  UPDATE payments SET status='success', paid_at, fee_kobo, channel
9  INSERT INTO ledger_entries (credit, gross      , athlete's wallet)
10 INSERT INTO ledger_entries (debit , provider fee, athlete's wallet)
11 identity.markPaid(verificationId, paymentId, tx) → under_review
12 INSERT INTO audit_logs
13 UPDATE webhook_events SET processed_at = now(), status='ok'
14 COMMIT
15 after commit: notify.enqueue(SMS receipt → athlete)   // never inside the tx
16 return 200

```

#### THE THREE LINES THAT WOULD COST YOU MONEY IF OMITTED

**Line 2** — without signature verification, anyone who learns the URL can grant themselves verification by POSTing a JSON body. **Line 3** — without the idempotency insert, a retried webhook writes the ledger twice and the books are wrong forever. **Line 7's amount check** — without it, a ₦10 payment against a ₦2,500 verification is approved, and that is the first thing anyone probing a Nigerian payment integration tries.

#### WHY THIS IS PROCESSED INLINE RATHER THAN QUEUED

Steps 4–14 take about 10ms. Paystack's timeout is generous and the persist-first insert at step 3 already means a crash after receipt loses nothing — the worker's reconciler (\$5) picks up any webhook row with a null `processed_at`. Queueing would add a moving part for no benefit at this volume. **If webhook processing ever exceeds one second, flip to: persist, return 200, process in the worker.** The code is already shaped for it because step 3 is separate from step 4.

#### PHASE 3 — REVIEW

The verification appears in the coordinator's queue. The reviewer sees the documents through short-lived presigned URLs (never public ones), and a checklist that includes the 18+ check. They approve or reject with a mandatory reason. A reviewer cannot act on their own record or on an athlete of a club they administer — checked in `identity.decide()`, not in the UI. On approval the photo becomes public and the profile gains its badge; on rejection the athlete may resubmit up to three times against the same payment.

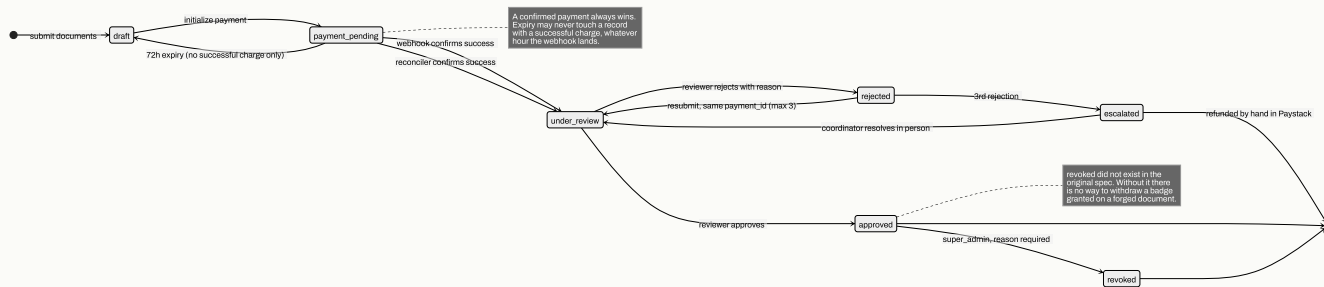


FIGURE 2 — VERIFICATION STATE MACHINE, INCLUDING THE TWO STATES ADDED IN §1 (REVOKED, ESCALATED)

### c) Assisted payment by an LGA coordinator

An athlete with no card hands ₦2,500 in cash to a coordinator, who pays on their own phone. Mechanically it is flow (b) with two fields set differently — and operationally it is the riskiest thing in the pilot, so the controls matter more than the code.

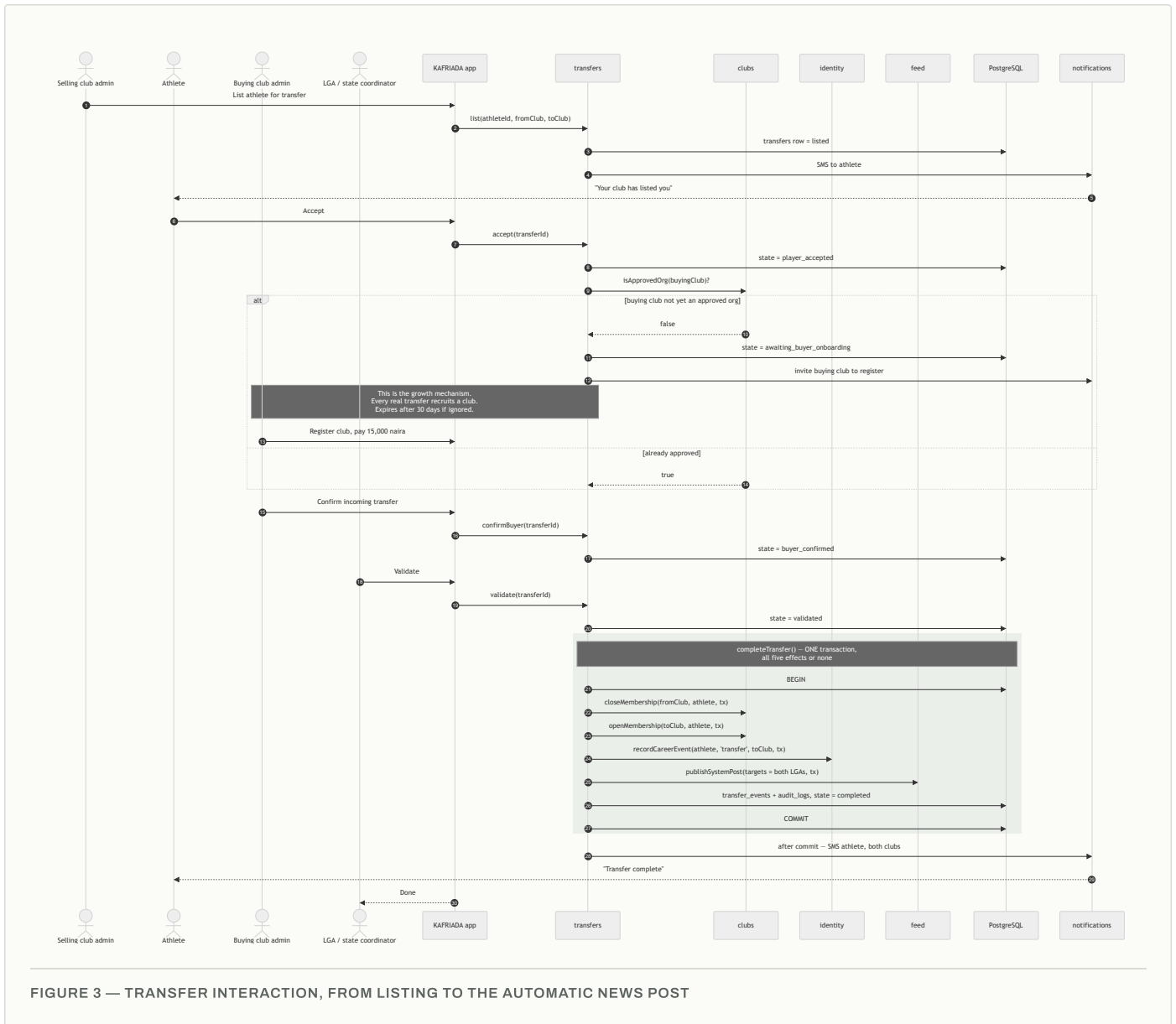
- 1 Coordinator logs in. Their session is `lga_coordinator`, scoped to one LGA — they cannot see or act on athletes outside it.
- 2 They search by KUID or phone. The athlete must be in their LGA; anything else returns nothing, not "access denied", so the search cannot be used to enumerate athletes elsewhere.
- 3 **Cap check.** Per coordinator, per day: 20 payments and ₦100,000. Exceeding it blocks and notifies the state coordinator. This bounds both fraud and card-blocking, and it is a table row you can tune per coordinator without a deploy.
- 4 They confirm the athlete's name and KUID on screen and tap "Pay on behalf". The screen shows exactly what will be charged and to whom it is credited.
- 5 `wallet.initiate()` creates the payment with `payer_user_id = coordinator`, `beneficiary_athlete_id = athlete`, `coordinator_id = coordinator`, `channel = assisted`. **The wallet and ledger entries belong to the athlete.** The coordinator is metadata on the payment, never a party to the ledger.
- 6 Coordinator pays with their card on Paystack. Webhook path is byte-for-byte flow (b) — the same function, no branch.
- 7 **On confirmation, two SMS.** To the athlete: KUID, ₦2,500 received, timestamp, coordinator's name. To the coordinator: a confirmation with the same reference. The athlete's SMS is the receipt that makes cash-skimming visible, and it is the single most valuable control in this flow.
- 8 The payment appears immediately on the coordinator's settlement report and on the state coordinator's daily reconciliation view: naira collected in cash, naira paid to Paystack, and the difference.

#### WHAT ARCHITECTURE CANNOT FIX HERE

Nothing above stops a coordinator taking cash and simply not paying. The SMS receipt only helps an athlete who notices its absence and knows where to complain. **This flow needs an operations control the software cannot provide:** a printed receipt book, a posted complaint number in every LGA, and a spot-check where the state coordinator calls five random athletes per LGA per week. Budget for it in the rollout, not in the sprint.

### d) A complete transfer

No money moves. The commercial value is entirely in the recruitment gate at step 4 — the buying club must join KAFRIADA to confirm — and the record it leaves behind.



Three details that are easy to get wrong:

- **The five completion effects are one transaction.** A partial completion — new membership opened, old one still open — produces an athlete at two clubs, and the partial unique index in `clubs` will refuse it. Good: the database catches the bug rather than the public profile displaying it.
- **The news post is generated, not written.** It is factual, so it auto-publishes rather than queueing for approval, but it carries `author = system` so an admin can find and remove one if a transfer is later reversed.
- **Notifications are sent after commit.** Telling an athlete their transfer completed and then rolling back is worse than telling them late.

## e) A QR scan by someone with no account

The highest-volume request in the system by a wide margin, and the only one a stranger ever makes. It should never touch your origin.

- 1 A scout scans the printed QR. It resolves to `https://kafriada.ng/a/KA-NG-JG-BKD-2026-000123?s=1.a1b2c3d4e5f60718`.
- 2 The request lands at **Cloudflare's Lagos PoP**. If the page is in cache — it will be, for any athlete scanned more than once in 60 seconds — it is served from Lagos and the origin never hears about it. **The cache key includes the query string**, so a signed and an unsigned request are different objects and a ticked page can never be served to an unsigned URL.
- 3 On a miss, Frankfurt renders the page server-side: one indexed lookup on `athletes.kuid`, one on career events, one on current club.
- 4 The signature is recomputed as `HMAC-SHA256(kuid, QR_SECRET[version])` and compared in constant time. Valid → "Issued by KAFRIADA" with a tick. Missing or invalid → the profile still renders, without the tick. **An invalid signature is not an error page** — a stranger typing a KUID by hand should still see the athlete.

- 5 Photo rendering is decided by verification state, not by the URL: `approved` shows the photo via a signed media path; anything else shows a silhouette. This check is server-side. A private R2 bucket means there is no second way to reach the image (§1.A.9).
- 6 Response carries `Cache-Control: public, max-age=60, stale-while-revalidate=600`. A verification approved at noon shows publicly within a minute — fast enough for a match-day dispute, long enough that a thousand scans of one athlete cost one origin request.
- 7 No cookie is set. No account is required. No personal data beyond the public profile is exposed — **never the phone number, never the date of birth, never the document**. Only name, KUID, LGA, sport, position, current club, career history, and photo if verified.

#### SAY WHAT THE TICK MEANS, ON THE PAGE

A copied URL keeps its valid signature (§1.A.10). The page should read *"Issued by KAFRIADA — check the photo matches the person in front of you"*, not *"Verified athlete"*. The honest wording costs nothing and is the difference between a scout who understands the tool and a scout who was misled by it.

## 05 Failure, scale and observability

Where it breaks first, what happens when each dependency dies, and the three screens you actually open every morning.

### Where this system breaks first

Not the database. At 5,000 athletes the whole dataset fits in RAM several times over, and §2 shows why volume is not the risk. The bottleneck is CPU, and it has a name.

#### BOTTLENECK #1 — ARGON2ID ON 2 VCPU

Argon2id is *designed* to be expensive: at OWASP's minimum parameters (19 MiB memory, 2 iterations, 1 degree of parallelism) a single hash costs roughly 50–100ms of dedicated CPU. On a 2 vCPU box that is a hard ceiling of **20–40 hashes per second across the entire application**, and every one of them is synchronous CPU work competing with your Node event loop.

Registration drives and login storms happen at the same moment — a rally where 200 people register while 30 coordinators log in. Nothing else in the system comes close to this cost. **Mitigations, in order:** (1) run hashing in a worker thread pool so it cannot block the event loop; (2) cap concurrent hashes with a semaphore at `vCPU count`, queueing beyond it, so latency degrades gracefully instead of the box falling over; (3) measure the real parameter cost on your actual instance in week 2 and tune memory cost down only if measurement demands it — never below OWASP minimums; (4) if you outgrow it, vertical scale is the answer, not a second instance.

Then, in order of when you would hit them:

| # | BOTTLENECK                  | CEILING         | WHAT YOU DO ABOUT IT, AND WHEN                                                                                                                                                                                                                    |
|---|-----------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | Image derivative generation | ~2/s in-process | Already solved by design: it runs in the worker, never in a request. If it were in-process it would be bottleneck #1.                                                                                                                             |
| 3 | Database connection pool    | ~20 connections | Fixed pool of 10 per process, pgBouncer in transaction mode in front. Node is not serverless here, so connection storms are avoidable rather than inherent — but pin the pool size explicitly or a deploy with two instances silently doubles it. |
| 4 | KUID counter row            | ~330–1,000/s    | Nothing, for years. Adding states divides it further (§4a).                                                                                                                                                                                       |
| 5 | Origin bandwidth on media   | n/a             | Nothing: media is served from R2, and profile pages from Cloudflare's edge. This is why the edge cache is a scale decision rather than a nicety.                                                                                                  |

### What happens when each dependency dies

The column that matters is the middle one. If a coordinator in Birnin Kudu cannot tell the difference between "our system is broken" and "Paystack is having a bad hour", they will tell athletes the wrong thing and you will lose trust you cannot buy back.

| DEPENDENCY        | WHAT THE USER SEES                                                                                                                                                            | WHAT THE SYSTEM DOES                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paystack down     | "Payments are temporarily unavailable — your submission is saved and you can pay when this clears." Registration, profiles, rosters, transfers and the feed all keep working. | Initialise calls fail fast with a 3s timeout and one retry. The verification stays in <code>draft</code> , never <code>payment_pending</code> , so the 72-hour clock never starts on an outage. Webhooks queued on Paystack's side arrive later and are processed normally. Alert fires after 3 consecutive failures.                                                                                    |
| Termii down       | Slower OTP. "Code sent — it can take a few minutes." A "resend" appears after 60s.                                                                                            | All SMS goes through <code>notifications_outbox</code> , so nothing is lost — the worker retries with exponential backoff for 24 hours. <b>This is why the outbox table exists</b> and why it survived my cut list. Registration is <i>not</i> blocked on OTP delivery: the record and KUID already exist; only the phone-verified flag waits. Alert when delivery rate drops below 90% over 15 minutes. |
| R2 down           | Uploads fail with "we could not accept your photo right now, try again shortly." Existing photos may fail to load; the silhouette shows instead. Everything else works.       | Presign calls fail fast. No half-created <code>media_assets</code> row — the row is written only after the client reports a successful PUT and the app HEADs the object to confirm it exists. Verification submission without a photo stays in <code>draft</code> .                                                                                                                                      |
| Database failover | 30–60 seconds of "we are having a moment, please try again". Then normal.                                                                                                     | In-flight transactions roll back — which is exactly why every money path is one transaction and why retries are idempotent. Connection pool reconnects to the new primary. The reconciler catches any payment whose webhook arrived during the window with a null <code>processed_at</code> . <b>The failover drill is part of the restore rehearsal, not a separate exercise.</b>                       |
| Cloudflare down   | The site is unreachable. Nothing you can do about it in this budget.                                                                                                          | Accepted risk, stated explicitly rather than pretended away. Keep the origin reachable on a secondary hostname known to the ops team so you can point DNS elsewhere in an extended outage. Do not build multi-CDN for a pilot.                                                                                                                                                                           |

| DEPENDENCY        | WHAT THE USER SEES                                                                | WHAT THE SYSTEM DOES                                                                                                                                                                                                                                                                                          |
|-------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App instance dies | Brief errors, then normal. Public profiles keep serving from the edge throughout. | Host restarts on health-check failure. Sessions live in Postgres, so no one is logged out. This is the argument for database-backed sessions over in-memory ones, and it is worth more than the small query cost.                                                                                             |
| Worker dies       | Nothing, for a while. SMS queues up; verifications do not expire on time.         | Jobs are idempotent and driven by table state rather than by a schedule, so a worker that has been dead for six hours catches up when it returns rather than skipping work. Alert if no job has completed in 15 minutes — <b>a silently dead worker is the failure most likely to go unnoticed for a day.</b> |

Reconciliation: when Paystack and your database disagree

Hourly, not daily. Every payment still `initialized` or `pending` more than one hour after creation is checked against `GET /transaction/verify/{reference}`.

| PAYSTACK SAYS                                | WE SAY                   | ACTION                                                                                                                                                       | WHY                                                                                                                                                                                                                      |
|----------------------------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>success</code>                         | <code>pending</code>     | <b>Apply the payment</b> by calling the identical function the webhook calls, guarded by the identical idempotency key. Alert as a webhook-delivery failure. | The customer paid. Reconciliation and the webhook must share one code path, or you will have two subtly different definitions of "paid" and eventually a bug that only appears in one of them.                           |
| <code>failed</code> / <code>abandoned</code> | <code>pending</code>     | Mark the payment failed. Leave the verification in <code>payment_pending</code> until the 72-hour expiry does its job.                                       | People abandon a checkout and come back an hour later. Do not slam the door early.                                                                                                                                       |
| <code>not found</code>                       | <code>initialized</code> | Mark <code>orphaned</code> , alert. Almost always means our initialise call never reached Paystack.                                                          | No money is involved, but a rising orphan count is an early signal of a network or credentials problem.                                                                                                                  |
| amount mismatch                              | any                      | <b>Freeze. Alert immediately. A human decides.</b> The job may not resolve this under any circumstance.                                                      | An amount mismatch is either an integration bug or an attack. Both deserve a person. An automated system that "fixes" money discrepancies is a system that can be manipulated into fixing them in someone else's favour. |
| <code>success</code> , already applied       | <code>success</code>     | No-op. Log at debug.                                                                                                                                         | The common case. The idempotency key makes it free.                                                                                                                                                                      |

THE RULE THAT KEEPS THE RECONCILER SAFE

The reconciler may only ever move money one direction: it can confirm a credit that already happened. It can never reverse, refund, debit or cancel. Every reversal is a human action recorded through `wallet.recordReversal()`. This single constraint means the worst a reconciler bug can do is credit someone twice — which the idempotency key already prevents and the nightly balance check would catch — rather than silently taking money away from people.

Two more jobs, both idempotent and both driven by table state:

- Daily expiry** — `payment_pending` records older than 72 hours *with no successful charge* return to `draft`. The "no successful charge" clause is \$1.A.5 made into code.
- Nightly integrity check** — for every wallet, credits minus debits must equal the cached balance; every athlete must have exactly one KUID; no KUID may appear twice; every `media_assets` row must point at an object that exists in R2. Any failure pages immediately. **This job is your smoke detector**, and it should be written in week 2, not week 7.

Observability from day one

WHAT CARRIES A TRACKING ID

Every request gets a `request_id` at the edge. It goes into every log line, onto every Sentry event, into the `meta` of every audit row written during that request, and — the part people skip — **onto the error screen the user sees**: "Something went wrong. Reference: 7f3a-91cc." A coordinator reading that down the phone turns a twenty-minute investigation into a thirty-second one, and it costs one line in an error component.

WHAT WE LOG, AND WHAT WE MUST NEVER LOG

LOG

Auth events including failures. Every permission denial with the permission that was missing. Full payment lifecycle. Every state transition, from and to. Webhook receipt, signature result and processing outcome. Every admin action. Every job run with its counts. Slow queries above 200ms.

## NEVER LOG

Passwords or hashes. OTP codes. Session tokens. Card data — you never receive any, and the log is the one place it could accidentally appear via a webhook payload dump. Full phone numbers (mask to +234801\*\*\*4567 ). Identity document contents. Add all of these to Sentry's scrubbing list, not only to your own logger.

## WHAT PAGES SOMEONE, AND WHAT MERELY OPENS A TICKET

| SEVERITY | CONDITION                                                                    | WHY THIS LEVEL                                                                                        |
|----------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| PAGE     | Nightly integrity check fails (ledger does not balance, or a duplicate KUID) | The two invariants the whole product rests on. Nothing else in this table matters if these are wrong. |
| PAGE     | Webhook processing failure, or any webhook unprocessed for 10 minutes        | Someone has paid and does not have what they paid for.                                                |
| PAGE     | Payments unresolved > 1 hour, count rising over two consecutive checks       | Distinguishes a blip from a broken integration.                                                       |
| PAGE     | 5xx rate above 2% over 5 minutes, or health check failing                    | The site is down for a meaningful share of users.                                                     |
| PAGE     | Any payment amount mismatch                                                  | Bug or attack; both need a person now.                                                                |
| TICKET   | SMS delivery below 90% over 15 minutes                                       | Degrades onboarding; the outbox means nothing is lost.                                                |
| TICKET   | Verification queue oldest item older than 18 hours                           | Early warning against the 24-hour pilot gate.                                                         |
| TICKET   | No worker job completed in 15 minutes                                        | A dead worker is invisible until something else breaks.                                               |
| TICKET   | Backup job failed, or database CPU above 70% for an hour, or disk above 80%  | Tomorrow's incident, today.                                                                           |
| TICKET   | Repeated invalid webhook signatures                                          | Either a misconfiguration or someone probing.                                                         |

## THE THREE DASHBOARDS THE CTO OPENS EVERY MORNING

Deliberately three. A fourth dashboard is a dashboard nobody reads.

### 1 — MONEY

Payments initiated / succeeded / failed in the last 24 hours, with success rate. Naira collected gross, fees paid, net. **Count of unresolved payments** and the age of the oldest. Assisted payments by coordinator. Ledger balance check: pass or fail.

*The one number: unresolved payments. If it is zero, no one is missing what they paid for.*

### 2 — FUNNEL

Registrations per day, split by LGA. Stage-1 → Stage-2 conversion against the 4% gate. Verification queue depth and oldest item age against the 24-hour gate. Clubs registered and rosters filled. Transfers by state, including how many are stuck in `awaiting_buyer_onboarding`.

*The one number: conversion rate. It is the pilot's commercial verdict.*

### 3 — HEALTH

p95 and p99 latency by route group. 5xx rate. Edge cache hit ratio. Webhook lag. SMS delivery rate. Database CPU, connections and slowest queries. Worker last-success time.

*The one number: p95 latency. It moves before anything else does.*

## THE TOOLING, KEPT BORING

**Sentry** for errors and performance traces. **Metabase** pointed at a read-only Postgres user for Money and Funnel — those are SQL questions, and SQL is the right tool. **The host's built-in metrics** for Health. No time-series database, no agent, no collector. Total added cost: near zero. Total added operational surface: one container.

## Backups, and what the restore drill must actually prove

An untested backup is a rumour. The drill happens once before launch, with a written result, and it is not "we restored it and the app started".

### THE RESTORE DRILL PASSES ONLY IF ALL EIGHT ARE TRUE

1. A **new** database was created from the backup *without touching production*. If the procedure requires stopping production, the procedure is wrong.
2. Recovery was to a **specific timestamp** — "ten minutes before we deliberately broke it" — not just to the last nightly snapshot. This is what makes RPO 5 minutes rather than 24 hours, and it is the difference between losing a morning's registrations and losing none.
3. Row counts match for `users` , `athletes` , `payments` , `ledger_entries` and `audit_logs` .
4. **The ledger balances in the restored database.** Credits minus debits equals the cached balance for every wallet — the nightly integrity job, run against the restore.
5. Every athlete has exactly one KUID, and no KUID appears twice.
6. The application boots against the restored database, a real login succeeds, and a real public profile renders with its photo — which proves that **R2 and the database still agree**. Object storage is not in a Postgres snapshot, and a restore that rolls the database back past an image upload leaves rows pointing at objects that exist, or objects pointing at rows that do not. R2 versioning with 30-day non-current retention is what makes this recoverable, and this step is what proves it.
7. The **wall-clock time was written down**. That number is your RTO. If nobody timed it, the drill failed regardless of the outcome.
8. The runbook was followed by **someone who did not write it**, from the document alone, with no one helping. This is the step teams skip and the one that determines whether the backup is useful at 3am when one person is unreachable.

## 06 Security floor

6,750 people will hand this system their identity and their money.  
The floor is not negotiable; everything above it is.

### The permission model

Eight roles live at launch. Permissions are rows, so the GDOC's remaining twenty are future `INSERT`s. Three properties make this safe rather than merely flexible:

1. **Deny by default, and fail closed.** Every route declares the permission it requires. A route with no declaration returns 403 at runtime *and fails the build* — because a route that is merely forgotten is exactly the route an attacker finds.
2. **Scope is checked with the permission, never after it.** `rbac.can(userId, 'club.roster.manage', {type:'club', id:42})`. A club\_admin has the permission for *their* club and no other. This is the single most common data-leak bug in role-based systems, and it is why scope is in the first migration (§1.C.3).
3. **Middleware, never UI.** Hiding a button is a courtesy to the user. The check happens on the server, on every request, including the ones the UI never issues.

| ROUTE GROUP                  | PUBLIC                                                                                                                     | ATHLETE | CLUB_ADMIN        | COACH    | SCOUT         | LGA_COORD | STATE_COORD | SUPER_ADMIN |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------|-------------------|----------|---------------|-----------|-------------|-------------|
| GET /a/{kuid}                | YES                                                                                                                        | YES     | YES               | YES      | YES           | YES       | YES         | YES         |
| Register / OTP               | YES                                                                                                                        | —       | —                 | —        | —             | —         | —           | —           |
| Own athlete profile          | NO                                                                                                                         | SELF    | NO                | NO       | NO            | NO        | NO          | YES         |
| Submit verification          | NO                                                                                                                         | SELF    | OWN CLUB          | NO       | NO            | NO        | NO          | NO          |
| Review / decide verification | NO                                                                                                                         | NO      | NO                | NO       | NO            | OWN LGA   | OWN STATE   | YES         |
| Manage club roster           | NO                                                                                                                         | NO      | OWN CLUB          | READ OWN | NO            | READ LGA  | READ STATE  | YES         |
| List athlete for transfer    | NO                                                                                                                         | NO      | OWN CLUB          | NO       | NO            | NO        | NO          | NO          |
| Accept transfer              | NO                                                                                                                         | SELF    | NO                | NO       | NO            | NO        | NO          | NO          |
| Confirm as buying club       | NO                                                                                                                         | NO      | OWN, APPROVED ORG | NO       | NO            | NO        | NO          | NO          |
| Validate transfer            | NO                                                                                                                         | NO      | NO                | NO       | NO            | OWN LGA   | OWN STATE   | YES         |
| Search athletes              | NO                                                                                                                         | NO      | OWN CLUB          | OWN CLUB | VERIFIED ONLY | OWN LGA   | OWN STATE   | YES         |
| Pay on behalf                | NO                                                                                                                         | NO      | NO                | NO       | NO            | OWN LGA   | NO          | NO          |
| Publish feed post            | NO                                                                                                                         | NO      | NO                | NO       | NO            | OWN LGA   | OWN STATE   | YES         |
| Settlement report            | NO                                                                                                                         | NO      | NO                | NO       | NO            | OWN ONLY  | OWN STATE   | YES         |
| Manage users & roles         | NO                                                                                                                         | NO      | NO                | NO       | NO            | NO        | NO          | RE-AUTH     |
| Revoke verification          | NO                                                                                                                         | NO      | NO                | NO       | NO            | NO        | NO          | RE-AUTH     |
| Record reversal              | NO                                                                                                                         | NO      | NO                | NO       | NO            | NO        | NO          | RE-AUTH     |
| Read audit log               | NO                                                                                                                         | NO      | NO                | NO       | NO            | NO        | OWN STATE   | YES         |
| Withdraw / send money        | No such route exists for any role, including super_admin. Not disabled — absent. A CI grep fails the build if one appears. |         |                   |          |               |           |             |             |

### HOW EVERY ROLE IS TESTED AGAINST EVERY ROUTE

The table above is not documentation — it is a fixture file checked into the repository. A generated test suite iterates every route in the route manifest against all eight roles plus anonymous, and asserts the expected status. Roughly **60 routes × 9 principals ≈ 540 assertions**, running in seconds on every pull request.

Two rules make it stay true: **a route not present in the fixture fails the build**, so a new endpoint cannot ship un-audited; and **the scoped cases are tested with a second tenant** — club\_admin of club A attempting club B's roster, coordinator of Birnin Kudu attempting a Dutse athlete. Testing only

"can club\_admin manage a roster" passes on a system that lets them manage *every* roster, which is the bug.

Rate limits, and where each is enforced

| LIMIT              | VALUE                                     | ENFORCED AT              | NOTES                                                                                                                               |
|--------------------|-------------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| OTP send           | 1 per 60s and 5 per day, <b>per phone</b> | Application (Postgres)   | Must be per target, not per IP — a hall of athletes shares one hotspot and would rate-limit each other.                             |
| Login              | 10 per hour per account <i>and</i> per IP | Application + Cloudflare | Per-account stops credential stuffing on one victim; per-IP stops broad spraying. You need both.                                    |
| Payment initialise | 10 per day per user                       | Application              | Coordinators get a separate, higher, per-coordinator cap (§4c).                                                                     |
| Registration       | 20 per hour per IP                        | Cloudflare               | Generous, because of shared connections at drives. Coarse filtering belongs at the edge, where it costs you nothing.                |
| Public profile     | Cloudflare defaults                       | Cloudflare               | Cached anyway; not worth an application check.                                                                                      |
| Webhook            | None                                      | —                        | Never rate-limit Paystack. Restrict by source IP allowlist instead — throttling a payment provider's retries is how you lose money. |

Application-level limits live in a Postgres `rate_limits` table with an upsert. **I cut Redis** — sessions are already database-backed, so there is no shared-state problem to solve, and adding Redis buys a cache nobody needs, a second thing to monitor, and a new failure mode. Revisit it if a single Postgres genuinely strains, which \$2 says will not happen before 10,000 athletes.

Secrets, transport, and the webhook's CSRF exemption

WHY THE WEBHOOK MUST BE EXEMPT FROM CSRF — AND WHAT THAT DOES NOT MEAN

CSRF protection defends a session-authenticated request from being forged by another site. The Paystack webhook has *no session*: it is a server-to-server POST from a company that has never seen your CSRF token. Applying CSRF to it would reject every real payment notification.

It is not unauthenticated, though — it is authenticated more strongly than a cookie, by an HMAC over the exact bytes of the body. Which produces the constraint people get wrong:

**READ THE RAW BODY BEFORE ANYTHING PARSES IT**

The signature covers the *bytes Paystack sent*. If a body parser deserialises the JSON and your code re-serialises it to check the HMAC, key order and whitespace change and the signature will never match — a bug that looks like "Paystack is sending bad signatures" and burns a day. In the App Router: `const raw = await req.text()` , verify, *then* `JSON.parse(raw)` . Never `req.json()` first. Pin `export const runtime = 'nodejs'` so `crypto` and the raw body are both available, and compare digests in constant time.

Defence in depth: allowlist Paystack's published source IPs at Cloudflare, so an attacker must both reach the route and forge an HMAC.

SECRETS

- Environment variables from the host's secret store. Never in the repository, never in the image, never in a `.env` that is committed. `gitleaks` runs as a pre-commit hook and again in CI — the hook catches honest mistakes, CI catches the machine without the hook installed.
- Separate values per environment. Staging must never hold a live Paystack secret key; a test key in staging means a mistake there costs nothing.
- **A written rotation procedure per secret, tested once.** `PAYSTACK_SECRET` and `QR_SECRET` matter most: Paystack's rotates in their dashboard and yours simultaneously (accept both briefly), and `QR_SECRET` rotates by *version*, which is why §1.A.10 puts a version in the signature. A secret you have never rotated is a secret you cannot rotate during an incident.
- Sentry and the logger get an explicit scrubbing list covering every field named in §5.

TRANSPORT AND SESSION

- Cloudflare **Full (strict)** with an origin certificate and authenticated origin pulls, so the origin refuses traffic that did not come through Cloudflare. Block direct-to-origin at the host firewall too.
- TLS 1.2 minimum. HSTS after a two-week soak; preload only once you are certain about subdomains.
- Session cookies `httpOnly; Secure; SameSite=Lax; Path=/'` . Lax rather than Strict because the Paystack callback is a cross-site top-level navigation back into your site and Strict would drop the session at the worst possible moment.

- Session rows in Postgres, revocable individually and in bulk. **Idle timeout: 30 minutes for staff roles, 30 days for athletes** — coordinators work on shared phones (\$1.A.15).
- Re-authentication for role changes, verification revocation and reversal recording.
- CSP with no `unsafe-inline`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, and a frame-ancestors deny.

NDPR

Nigeria's Data Protection Regulation applies from the first registration, and one of its obligations is quietly triggered by your own pilot target.

| OBLIGATION               | HOW IT IS MET IN THE PILOT                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Privacy notice at signup | Shown before submission, with an explicit checkbox. The consent is <i>recorded</i> — a <code>consents</code> row holding the notice version and a timestamp. "We showed them a link" is not evidence; a versioned row is. This is why <code>consents</code> survived the cut list.                                                                                                                                                                                                        |
| Lawful basis             | Consent for the profile; contract for the paid verification; legitimate interest for fraud prevention and audit. Written down once, in the notice, in plain language.                                                                                                                                                                                                                                                                                                                     |
| Data export on request   | Manual in the pilot: a <code>super_admin</code> action producing a JSON bundle of everything held about one subject, delivered by email, with an audit row. Automating it is Phase 2 — at four requests a pilot, a script is the right answer.                                                                                                                                                                                                                                            |
| Deletion on request      | <b>The conflict must be disclosed up front, not discovered at the first request.</b> Audit rows cannot be deleted, financial records carry a retention duty, and the KUID is immutable by design. So "deletion" means <i>anonymisation</i> : name, photo, phone, date of birth and documents are erased; the KUID shell, the ledger and the audit trail remain. <b>Say this in the privacy notice in plain words.</b> A promise you cannot keep is worse than a limitation you explained. |
| Retention                | Audit 7 years. Identity documents deleted from R2 30 days after a verification decision — you needed them to decide, not to keep. Media versioning 30 days.                                                                                                                                                                                                                                                                                                                               |
| Filing obligation        | <b>BUDGET ITEM</b> NDPR requires organisations processing more than 2,000 data subjects in twelve months to file an annual data-protection audit through a licensed compliance organisation. <b>Your pilot target of 5,000 athletes crosses that threshold.</b> This is not a code task — it is a line in the budget and a name in the plan, and it is the kind of thing that surfaces at the worst moment if nobody raises it now.                                                       |
| Breach response          | A one-page runbook: who is called, how affected subjects are identified from the audit log, what NITDA is told and within what window. One page written in week 7 beats ten pages written never.                                                                                                                                                                                                                                                                                          |

## 07 Production environment and rollout

What runs where, what it costs, and why the 27 LGAs go live in waves rather than all at once.

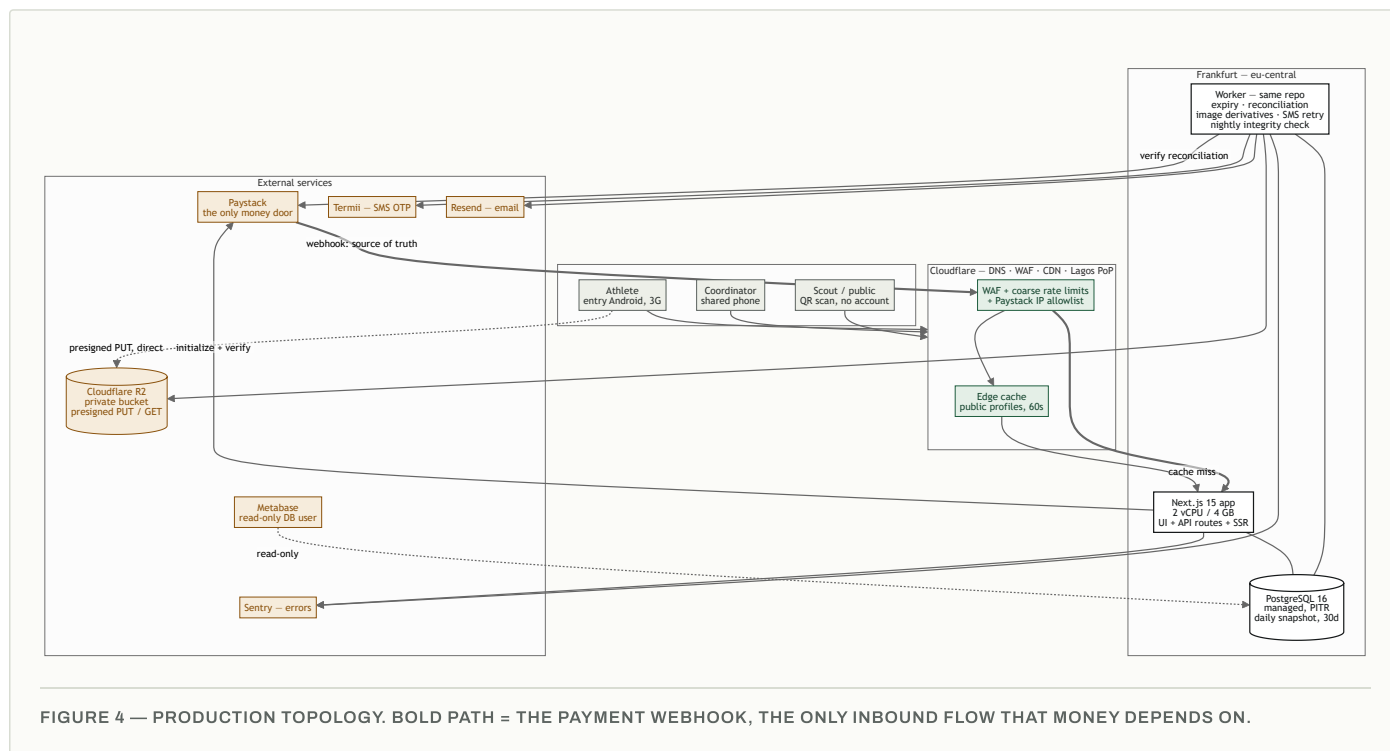


FIGURE 4 — PRODUCTION TOPOLOGY. BOLD PATH = THE PAYMENT WEBHOOK, THE ONLY INBOUND FLOW THAT MONEY DEPENDS ON.

### Environments and what each is for

- **Local** — Docker Compose: Postgres, MinIO standing in for R2, Paystack test keys, SMS written to a log file instead of sent. A developer must be able to run a complete payment on a laptop with no network dependency beyond Paystack's test mode.
- **Staging** — the same shape, one size smaller, same region, test keys throughout. This is where the load test and the dependency-failure drills run. It holds no real personal data, ever.
- **Production** — as drawn. Migrations run as a gated release step before the new version takes traffic; every migration must be backwards-compatible with the currently-running version, so a rollback does not require a database rollback.

### Monthly cost

| ITEM                                        | USD / MONTH   |
|---------------------------------------------|---------------|
| App instance — 2 vCPU / 4 GB                | 25–35         |
| Worker instance — small                     | 7             |
| Managed Postgres with PITR                  | 20–40         |
| Cloudflare Pro (WAF rules, analytics)       | 20            |
| R2 storage and egress at ~2 GB              | ~0            |
| Sentry, Metabase, Resend — free tiers       | 0             |
| Termii SMS — usage, roughly 12,000 messages | 25–40         |
| <b>Total</b>                                | <b>97–142</b> |

Against the \$60–120 budget: at the top end you are slightly over, and the overage is SMS, which is a variable cost that scales with success. Cloudflare Pro is the one item you could drop to hit the number, at the cost of managed WAF rules. **Do not save money on Postgres PITR** — that line item is your RPO.

Rollout: phased, by LGA

Three options were on the table. Taking them in turn:

- **Parallel run** is not meaningful — there is no existing system to run alongside. Discount it.
- **Full launch** across 27 LGAs on one date is reckless, and not because of the code. The untested component is *the operation*: coordinators handling cash, a verification queue meeting a 24-hour SLA, athletes who have never installed a PWA. Launching everywhere at once means discovering an operational flaw in 27 places simultaneously, with three developers.
- **Phased by LGA** is the recommendation.

RECOMMENDATION

Phase by LGA, gated on numbers rather than dates, using a `locations.is_live` flag so a wave can be opened or closed **without a deploy**. Registration is refused in an LGA that is not live; public profiles work everywhere from day one. The flag costs one boolean and buys you the ability to stop a bad wave in thirty seconds.

| WAVE               | SCOPE                      | DURATION  | WHAT IT IS ACTUALLY TESTING                                                                                                                                                                                                                                            |
|--------------------|----------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 — Internal       | Team + 20 invited athletes | 3 days    | Real money through Paystack in live mode, including one deliberate refund and one deliberate duplicate webhook. The restore drill runs here (\$5), before any member of the public is exposed.                                                                         |
| 1 — Birnin Kudu    | 1 LGA, ~300 athletes       | 2 weeks   | The soft launch. The whole operation with all three developers present and one of them physically at a registration drive watching people use it. <b>Assisted payment is the thing under test</b> — cash handling, receipts, settlement — and it needs eyes, not logs. |
| 2 — Anchor cluster | +3 neighbouring LGAs       | 2 weeks   | Whether the operation works when a developer is <i>not</i> in the room. The first real test of the runbook and the support line.                                                                                                                                       |
| 3 — Half the state | +11 LGAs (15 total)        | 2 weeks   | Verification throughput against the 24-hour SLA with a queue that is no longer trivially small. Coordinator training at scale.                                                                                                                                         |
| 4 — Full state     | +12 LGAs (27 total)        | remainder | Volume against the pilot gate: 5,000 registrations, 4% conversion, 81 active clubs.                                                                                                                                                                                    |

GO / NO-GO GATES BETWEEN WAVES

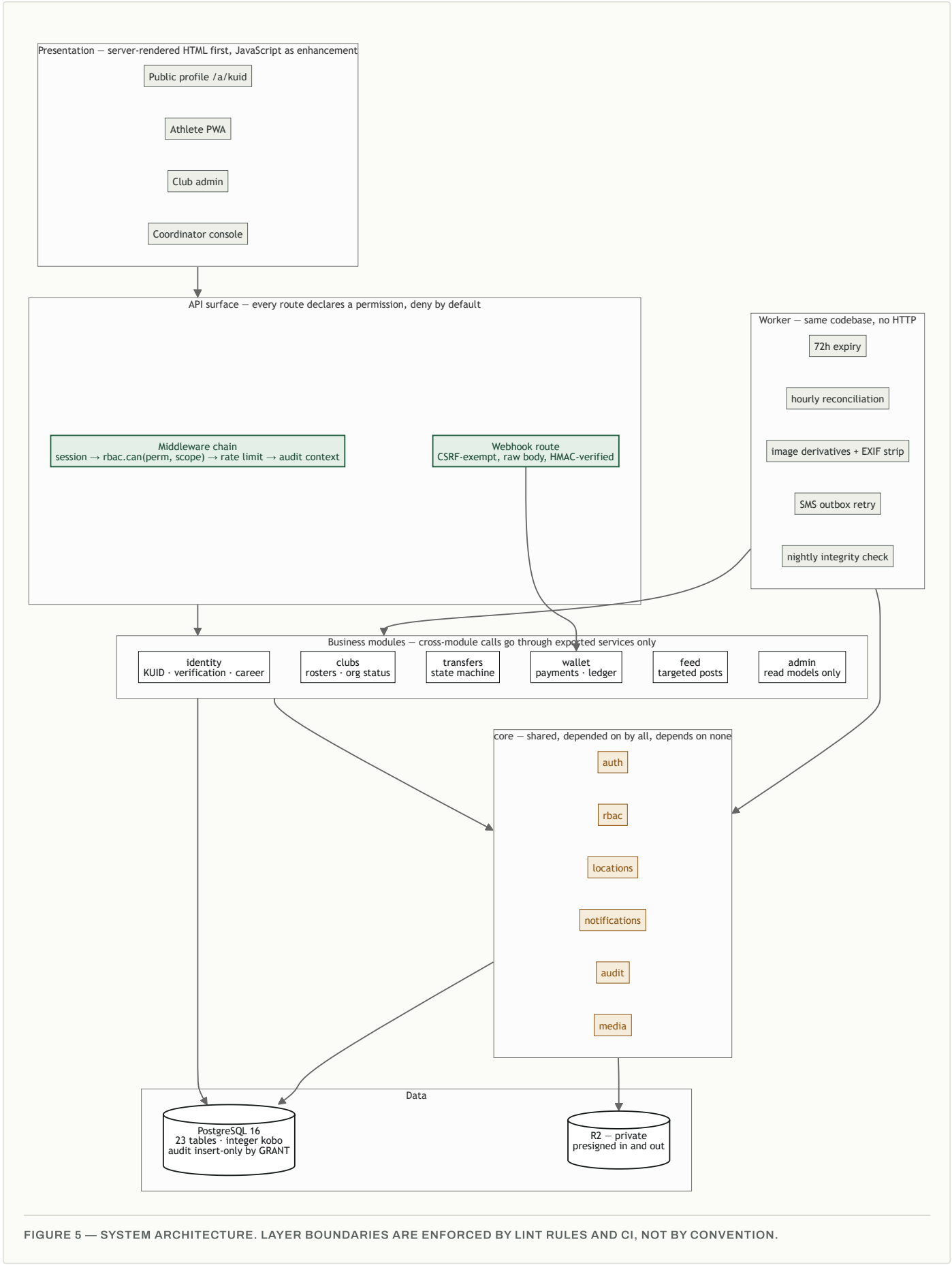
Numbers, decided in advance, so the decision to proceed is not made by whoever is most optimistic in the room that morning. All five must hold:

- Payment success rate **above 92%** of initialised transactions that reach Paystack's page.
- **Zero** payments unreconciled for more than 24 hours.
- Verification p50 **under 6 hours**, p95 under 24.
- Fewer than **3 support contacts per 100 registrations**.
- Zero unexplained ledger discrepancies — the nightly integrity check green every night of the wave.

A failed gate holds the wave. It does not roll back the previous one, because the previous LGAs have working records and pulling the service is worse than pausing expansion.

## The artifacts

The Solution Design Document outline, and the system-level architecture diagram.



Solution Design Document — outline

The document the CEO was promised. Everything in this artifact slots into it; the sections below are what a reviewer expects to find and in what order.

| §  | SECTION                         | CONTENTS                                                                                                                                                                                                                                                                                                                                    |
|----|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Business context and objectives | The identity wedge: free registration builds the register, paid verification monetises it, free transfers recruit clubs. Pilot gate — 5,000 registrations, 4% conversion, 81 clubs, verification under 24 hours. What success authorises next, and what failure stops.                                                                      |
| 2  | Scope                           | In: identity, verification, clubs, transfers as records, targeted feed, closed-loop ledger. Out and <i>sequenced, not cancelled</i> : medical records, marketplace, scout subscriptions, transfer money, native app, minors.                                                                                                                |
| 3  | Assumptions                     | Phone number anchors identity. Coordinators are trained and supervised. Paystack is available in Jigawa and settles to a Nigerian account. Athletes have SMS-capable phones, not necessarily smartphones. 18+ is self-declared and only checked for the 4% who pay. \$1.B in full.                                                          |
| 4  | Constraints                     | 8 weeks, 3 developers, \$60–120/month. No banking licence, therefore no payout path. NDPR, including the filing threshold. Entry-level Android and proxy browsers, therefore no-JavaScript-required. One state at launch.                                                                                                                   |
| 5  | Dependencies                    | Paystack (critical — no fallback, accepted), Termii (degraded-tolerant via the outbox), R2, Cloudflare, Resend, Sentry. For each: what breaks, what still works, §5's table.                                                                                                                                                                |
| 6  | Requirements                    | §2 verbatim — functional table, non-functional table with measurable targets, the scale table.                                                                                                                                                                                                                                              |
| 7  | Architecture                    | Stack decision and its justification (§0), module map and boundary enforcement (§3), Figures 1, 4 and 5.                                                                                                                                                                                                                                    |
| 8  | Detailed design                 | The five critical flows with the concurrency and idempotency arguments (§4), Figures 2 and 3, the data model, and the two state machines.                                                                                                                                                                                                   |
| 9  | Security and compliance         | §6 in full: permission matrix, its test strategy, rate limits, secrets, webhook CSRF exemption, NDPR obligations including the filing budget line.                                                                                                                                                                                          |
| 10 | Operations                      | §5: failure modes, reconciliation, alerting split by severity, the three dashboards, backup and the eight-point restore drill.                                                                                                                                                                                                              |
| 11 | Methodology and delivery        | Two-week sprints against the wave plan. Trunk-based with short branches. Every pull request runs the permission matrix, the burst test and Lighthouse. Sprint 0 is schema, RBAC, audit and the payment skeleton — <b>the money path is built first and hardened all the way through, because it is the part that cannot be fixed later.</b> |
| 12 | Risks and decisions             | The three expensive-to-change decisions (§1.C) with their owners and deadlines. The deferred GDOC features with the conditions that would bring each forward. The open questions below.                                                                                                                                                     |

## 09 What I cut, and what I need from you

Every cut has a reason and a trigger that would reverse it.

| CUT                                                       | WHY                                                                                                                                                                | WHAT WOULD BRING IT BACK                                                                                                  |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Redis                                                     | Sessions are already in Postgres, so there is no shared-state problem. Rate limits fit in a table. It would add a service to monitor and a new way to fail.        | Postgres write contention that profiling actually attributes to rate limiting — not before.                               |
| Fan-out notification tables for the feed                  | 27 LGAs and 5,000 users. Query on read with an index; the whole feed is a few thousand rows.                                                                       | Six figures of users, or per-user read state becoming a product requirement.                                              |
| A message bus / event system                              | Six modules in one process. Function calls are simpler, atomic with the transaction, and debuggable with a stack trace.                                            | Splitting a module into its own service — at which point the bus is the point, not an accessory.                          |
| Elasticsearch                                             | Postgres full-text search over 5,000 athletes is instant and free. <i>Deferred by CTO decision; I am not reopening it.</i>                                         | Hundreds of thousands of athletes, or fuzzy multilingual name search becoming a scouting feature people pay for.          |
| GraphQL                                                   | One client, one team, one deploy. It would add a schema layer and a new authorisation surface for no benefit. <i>Deferred by CTO decision.</i>                     | Third-party API consumers — federations, or a native app built by someone else.                                           |
| Microservices                                             | Three developers. The module boundaries in §3 give you the option later without paying for it now. <i>Deferred by CTO decision.</i>                                | Team growth past roughly a dozen developers, or one module needing to scale independently.                                |
| JWT in localStorage                                       | XSS-readable, hard to revoke, and there is nothing to be stateless <i>for</i> . <i>Deferred by CTO decision, and correctly so.</i>                                 | Nothing in the foreseeable roadmap. Database-backed sessions remain right even at Year-1 scale.                           |
| Multiple payment gateways                                 | Every gateway is a webhook contract, a reconciliation path and a set of failure modes. One is enough to get right in eight weeks. <i>Deferred by CTO decision.</i> | A Paystack outage long enough to cost a rollout wave, or settlement terms that make a second rail commercially necessary. |
| Native app                                                | The PWA reaches every Android in Jigawa without a store listing.                                                                                                   | A feature genuinely needing native capability — offline scanning at venues with no signal is the realistic candidate.     |
| Automated GDPR/NDPR export tooling                        | At an expected handful of requests, a script plus a person is the right engineering.                                                                               | More than roughly one request a week.                                                                                     |
| Kubernetes, service mesh, feature-flag SaaS, multi-region | Not one of these connects to a business goal or a non-negotiable at this scale. A boolean column is your feature flag.                                             | None of them, this year.                                                                                                  |

### What I added, and the argument for it

One addition to the Build Spec: **a second always-on worker process**, same repository, no HTTP surface. It is not an optional nicety. Reconciliation, the 72-hour expiry, image derivative generation, SMS retry and the nightly integrity check cannot run inside web requests — and if you try to run them from a request-triggered cron or a serverless timer, the first outage that spans a scheduled window silently skips work that touches money. Cost: about \$7 a month and one extra deploy target. **Everything in §5 that catches a money problem runs here.**

### Open questions — answer these and the design closes

None of these block the sprint. Each changes something material, and I have stated the assumption I am building on until you say otherwise.

| # | QUESTION                                                       | ASSUMED ANSWER I AM BUILDING ON                                                                                                      | WHAT CHANGES IF YOU SAY OTHERWISE                                                                                                                                                                                                            |
|---|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Are the three developers TypeScript-primary or Python-primary? | TypeScript. Next.js monolith as specified.                                                                                           | Everything in §0 flips to FastAPI + Next.js. The rest of this document is unchanged. This is the only question that touches the stack, so it is the one I want answered first.                                                               |
| 2 | Is ₦2,500 the price or the take?                               | The price. You receive about ₦2,362 after Paystack's fee.                                                                            | If the CEO's revenue model assumes ₦2,500 net, the price becomes ₦2,640 and every piece of published material changes. Ask before anything is printed.                                                                                       |
| 3 | Do we collect NIN?                                             | No. Phone number anchors identity; NIN is Phase 2.                                                                                   | Collecting it adds NIMC compliance duty, a verification cost per lookup, and encryption-at-rest requirements for the field. Do not collect it unverified.                                                                                    |
| 4 | Who approves verifications, and what is the conflict rule?     | LGA coordinators within their own LGA, escalating to state. A reviewer may never approve their own record or a club they administer. | If a central review team is preferred, the scope check changes but nothing else does. The conflict rule is not optional either way.                                                                                                          |
| 5 | What is the refund policy?                                     | Refunds are rare, manual, initiated in Paystack's dashboard by a super_admin, and mirrored into the ledger as a recorded reversal.   | If refunds are expected to be common, the settlement report and the coordinator caps both need rework — and the conversation about chargeback risk on assisted payment becomes urgent rather than precautionary.                             |
| 6 | What do coordinators actually carry, and which browser?        | Entry-level Android on 3G, some of it Opera Mini in proxy mode. Hence the no-JavaScript-required rule.                               | If they are all on modern Chrome, the constraint relaxes — but I would keep it anyway. It costs little and it is the difference between working and not for the athletes, who are a much larger and poorer population than the coordinators. |

| # | QUESTION                                      | ASSUMED ANSWER I AM BUILDING ON             | WHAT CHANGES IF YOU SAY OTHERWISE                                                                          |
|---|-----------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 7 | Who owns the NDPR filing, and is it budgeted? | Nobody yet. Flagged in §6 as a budget line. | Nothing technical. It is a name, a date and a cost — and it is the item most likely to be discovered late. |

THE THROUGH-LINE

Almost everything in this document is ordinary. One transaction that either happens completely or not at all. One unique index instead of a check in code. Reading a webhook's raw bytes before parsing them. Writing the restore time down. **None of it is clever, and that is the point** — 6,750 people are going to trust this with their identity and their money, and what earns that is a system whose failures are boring and whose records cannot be quietly altered. The interesting engineering here is deciding what not to build; the rest is doing the dull parts exactly right, the first time, because the money path and the KUID are the two things you do not get to fix later.